

Final Project

**FPGA Based Prime Number Guessing Game With ROM
Based Multi User Authentication And RAM-Score Tracking**

Team Name: ArithmeticAces

Dharani Kandharam – 2431389 – dkandhal@cougarnet.uh.edu

Sri Sai Dhanush Karnatapu – 2357472 - skarnata@cougarnet.uh.edu

Narendhar Puttinti – 2454090 - nputtint@cougarnet.uh.edu

Mukesh Musham – 2430886 - mmusham@cougarnet.uh.edu

Hysritha Sai Kasarapu – 2442341 - hkasarap@cougarnet.uh.edu

ECE6370: Advanced Digital Design

Contents

1. Introduction.....	4
1.1 FPGA Board:	5
1.2 Components used for this game on the Board:	6
2. System Architecture Design:	7
2.1 Top Level Block Diagram:	7
2.2 Multiuser Authentication	8
2.2.1 Authentication Module	8
2.2.2 ID_Checking_Module:	9
2.2.3 Password_Checking_Module:	10
2.3 ButtonShaper:	12
2.3.1 ButtonShaper Block Diagram:	12
2.3.2 ButtonShaper FSM:	12
2.3.3 ButtonShaper Timing Diagram	13
2.4 LevelSelector Module:	13
2.4.1 LevelSelector Module Block Diagram:	13
2.5 Game Controller:	14
2.5.1 PrimeGameController Module Block Diagram:	14
2.5.2 FSM PrimeGameController Module:	15
2.6 Prime Generation:	16
2.6.1 PrimeTargetGenerator Module:	16
2.6.1.2 FSM for PrimeTargetGenerator Module:	16
2.6.2 RandomIndexLFSR Module:	17
2.6.3 PrimeLFSR Module:	18
2.6.4 ROM_PrimeNumbers Module:	18
2.7 Guess Input:	19
2.7.1 TwoDigitGuessInput Module:	19
2.8 Prime Checking:	20
2.8.1 Prime Checking Module:	20
2.9 Timer System:	21
2.9.1 OneSecTimer:	21
2.9.1.2 OnemillisecLFSRTimer Module:	21
2.9.2 TwoDigitTimer:	22
2.9.3 ThreeDigitTimer Module:	23
2.10 ScoreBoard:	24

2.10.1 ScoreBoard Module Block Diagram:	24
2.11 Display System:	25
2.11.1 DisplayController:	25
2.11.2 7SegmentDecoder:.....	25
2.12 LoadRegister.....	26
2.13 Score Tracking and Winner Detection	27
2.13.1 Score Tracking and Winner Detection Block Diagram.....	27
2.13.2 Finite State Machine for Score Tracking Module:	28
3 Simulation Results:	29
3.1 7-Segment Decoder (My7SegmentDecoder_4to7) Simulation:.....	29
3.2 ButtonShaper Simulation:	31
3.3 LoadRegister Simulation:	32
3.4 OneSecTimer:.....	34
3.5 TwoDigitTimer:.....	37
3.6 Random Number Generator:.....	38
3.7 1ms LFSR Timer:.....	38
4 FPGA Board Testing Results:	42
5 Video Demo:.....	49
5.1 Player1 Demo Links:	49
5.2 Player2 Demo Link:	49
5.3 Global and local winners: Player2_Global_Local_winners.mp4.....	49
5.4 Difficulty levels: Different_Levels.mp4.....	49
6 Conclusion:	49
7 Appendix:	51

1. Introduction

This project implements a fully hardware-based Prime Number Guessing Game on the DE0-CV FPGA board using Verilog HDL. The system is designed as an interactive digital game in which authenticated players attempt to guess a hidden target prime number within a limited time interval. The complete design integrates multiuser authentication, pseudo-random prime-number generation, timer-controlled gameplay, score tracking, and real-time display management into a modular FPGA-based system architecture.

The game begins in the logged-out state after FPGA initialization or system reset. Before accessing gameplay functions, the player must complete a multiuser authentication process by entering a valid player ID and password through FPGA switches and push buttons. The entered credentials are verified using ROM-based lookup tables that store valid player IDs and passwords. Only after successful authentication does the system allow access to the game controller, timer system, and prime-number generation modules.

After authentication, the player selects a gameplay difficulty level using dedicated switches connected to the LevelSelector module. The selected level determines the gameplay timer duration and the range of prime numbers used during the game. Three gameplay levels are supported:

- Level 1: Smaller prime-number range with longer gameplay duration
- Level 2: Medium prime-number range with moderate gameplay duration
- Level 3: Full prime-number range up to 97 with shorter gameplay duration

Once the player starts the game, the PrimeGameController activates the timer system and initiates hidden target prime generation. The target prime number is generated using an LFSR-based pseudo-random index generator combined with a ROM-style prime-number lookup table. Instead of generating arbitrary numbers directly, the system generates pseudo-random indices that access stored prime-number values. Since every value stored in the ROM table is guaranteed to be prime, the generated target number is always a valid prime number.

The player enters guesses using FPGA switches and confirms the input using a LOAD button. The PrimeChecker module first verifies whether the entered number is itself a valid prime number using a ROM-based lookup method instead of computational algorithms such as division or modulus operations. If the entered value is not prime, the system displays a WRONG indication. If the entered value is prime, the game controller compares the guess with the hidden target prime and generates one of the following gameplay hints:

- TOO SMALL (TS)
- TOO LARGE (TL)
- CORRECT (C)

Whenever the player correctly guesses the hidden target prime number, the scoreboard increments the player's score and the system automatically generates a new target prime number for the next gameplay round. The game continues until the countdown timer expires.

The timer system is implemented using a hierarchical architecture consisting of LFSR-based millisecond timers, hundred-millisecond timers, and one-second timers combined with a configurable three-digit countdown timer. Different timer preset values are loaded depending on the selected gameplay difficulty level.

The DisplayController module manages all seven-segment display outputs used throughout the game. The displays dynamically present authentication status, gameplay hints, timer values, score values, and other gameplay information depending on the current operating state of the system.

The overall design demonstrates FPGA-based system integration involving ROM-based authentication, pseudo-random number generation using LFSRs, hierarchical timer systems, modular game control logic, prime-number lookup methods, score tracking, and real-time hardware interaction. The project emphasizes modular hardware design, efficient FPGA resource utilization, and non-computational prime-number processing techniques suitable for digital system implementation.

1.1 FPGA Board:

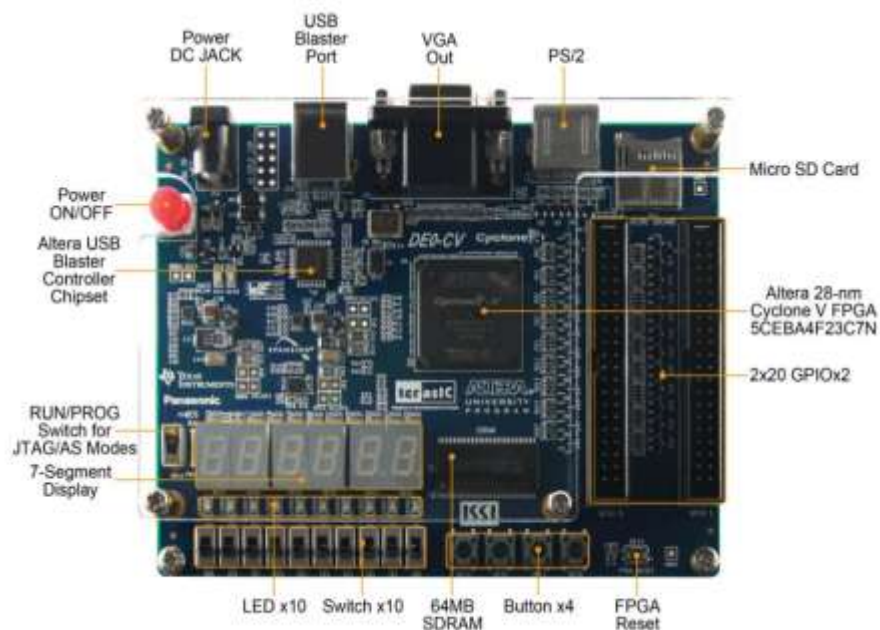


Figure1.1 : Development Board (top view)

1.2 Components used for this game on the Board:

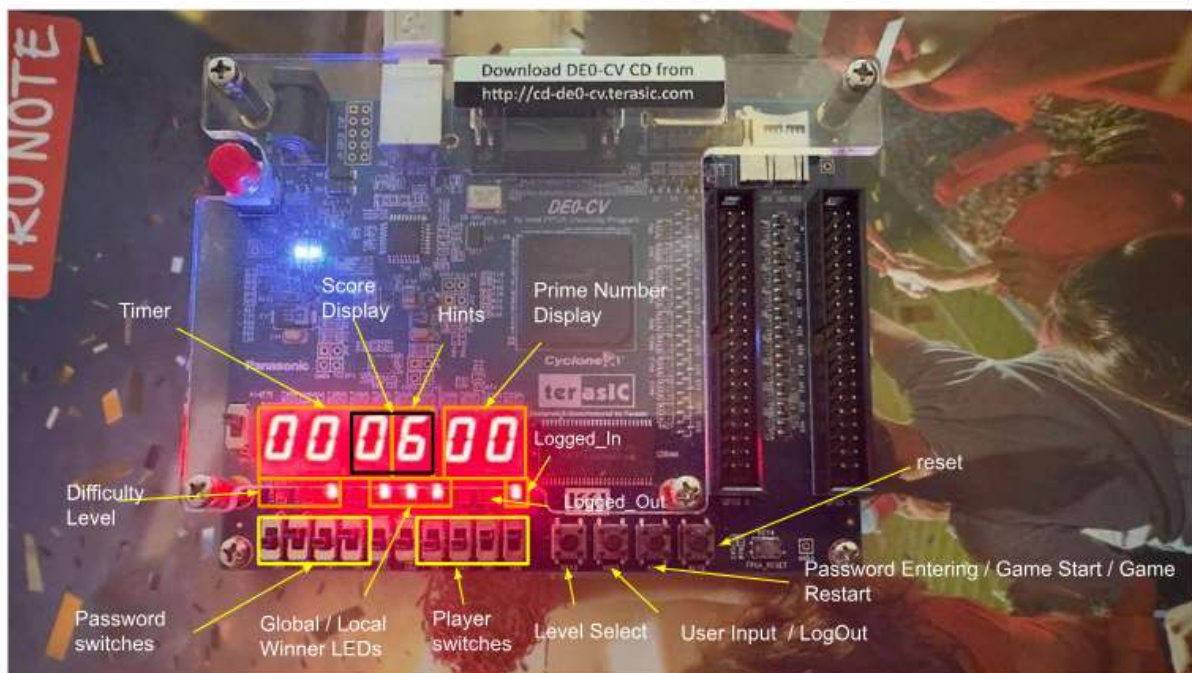


Figure1.2 : Components used for this game

2. System Architecture Design:

2.1 Top Level Block Diagram:

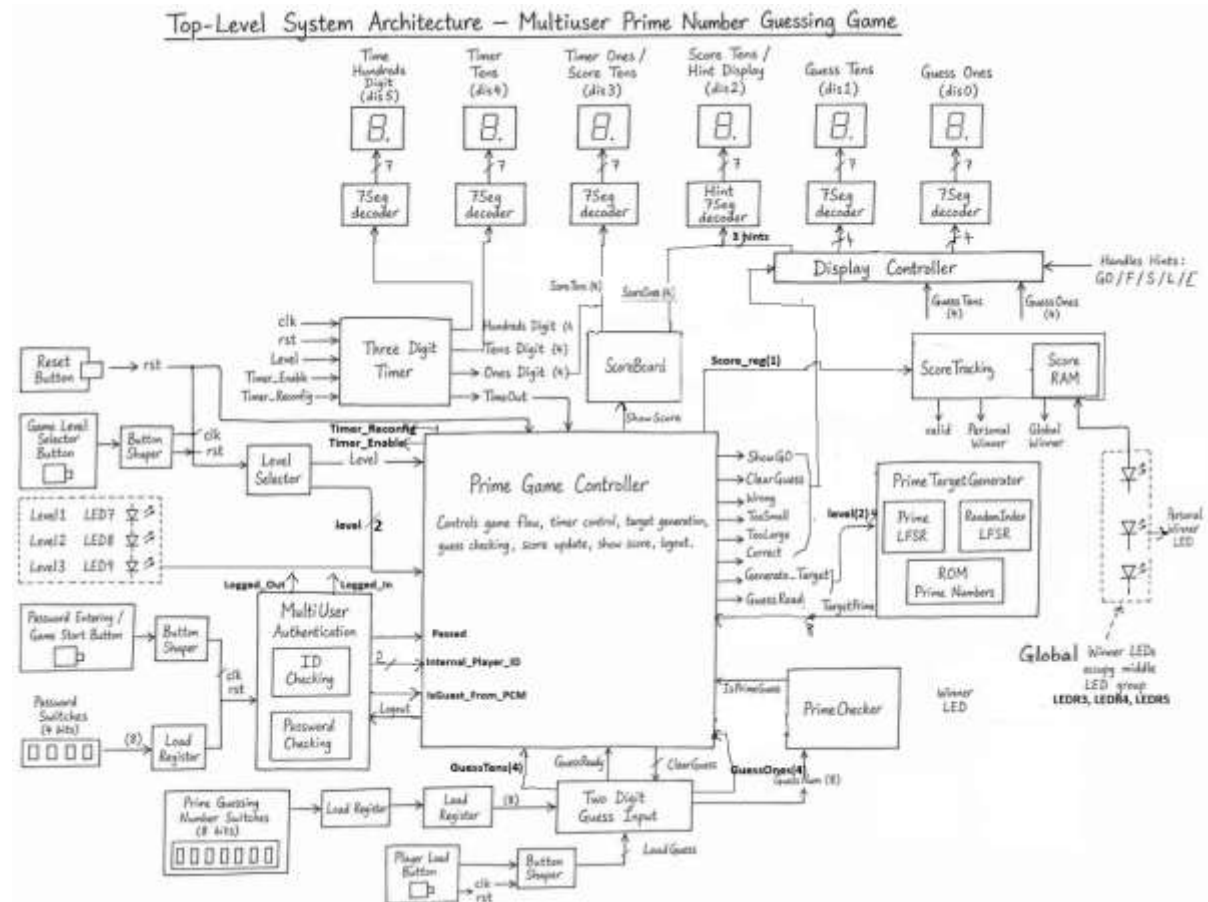


Figure2.1: Top-level system architecture

2.2 Multiuser Authentication

2.2.1 Authentication Module

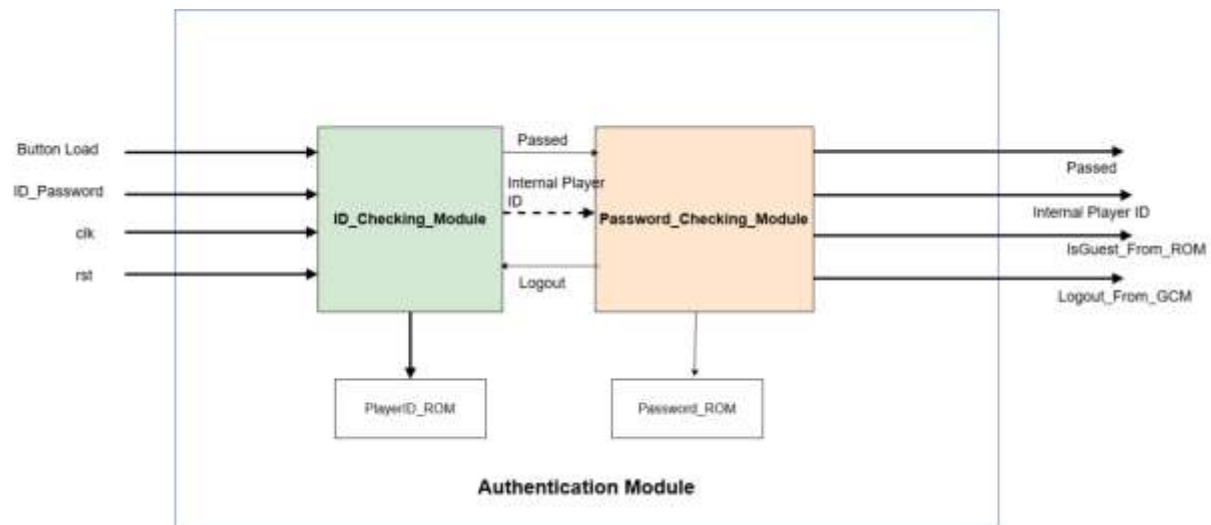


Figure 2.2.1: Authentication Module Block Diagram

Inputs: Button_Load ,ID_Password[3:0] ,clk ,rst

Outputs:Passed ,InternalPlayerID[2:0] ,IsGuest_From_ROM ,Logout_From_GCM

Description:

The Authentication Module controls secure multiuser access to the Prime Number Guessing Game by verifying both player identification and password credentials before gameplay is enabled. The module is divided into two primary stages: the ID_Checking_Module and the Password_Checking_Module.

During the first stage, the ID_Checking_Module validates the entered user ID against values stored in the PlayerID_ROM. Once a valid ID is detected, the module forwards the corresponding internal player identification value to the Password_Checking_Module.

The second stage authenticates the player password using values stored in the Password_ROM. If the password matches successfully, the authentication process is completed and the module asserts the Passed signal, allowing access to gameplay functions.

The module also generates:

- Internal player identification values used for score tracking and game control
- Guest-user information retrieved from ROM storage
- Logout control synchronization signals for system-level session handling

This modular authentication structure improves system security, supports multiuser gameplay capability, and simplifies integration with the PrimeGameController and score-tracking subsystem.

2.2.2 ID_Checking_Module:

2.2.2.1 ID_Checking_Module Block Diagram:

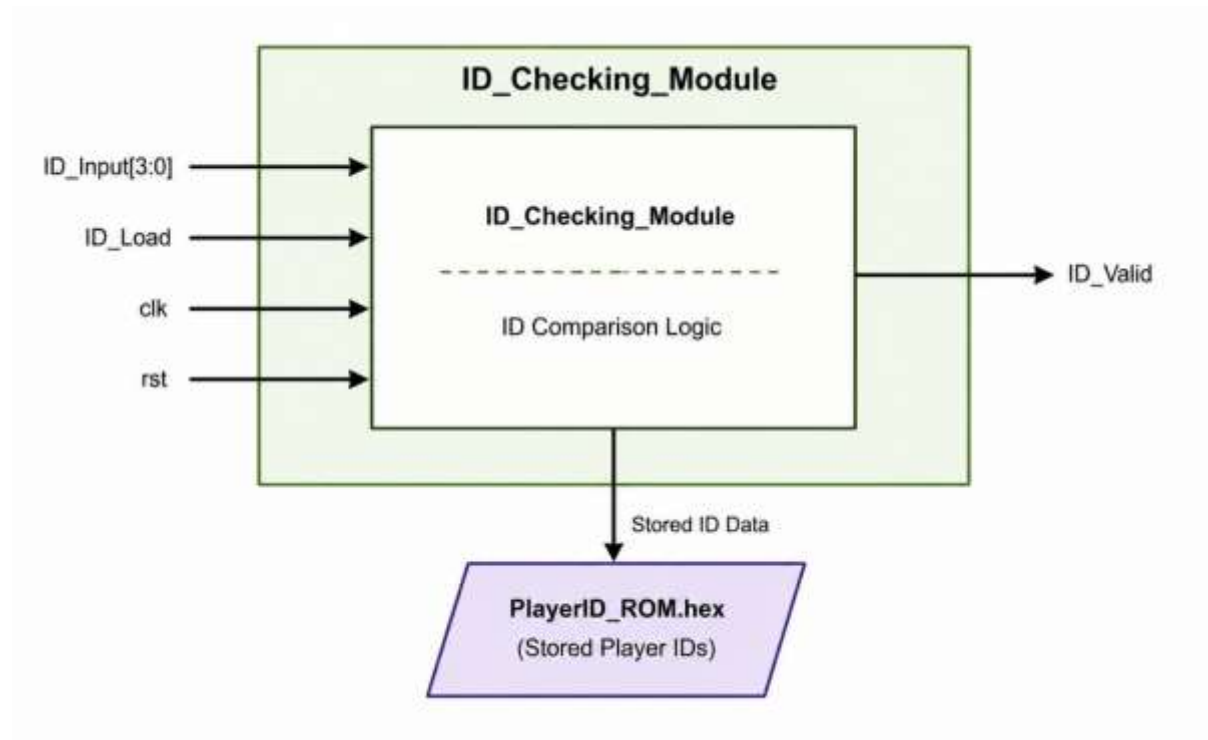


Figure 2.2.2.1: ID Checking Block Diagram

Inputs: ID_Input[3:0], ID_Load, clk, rst

Outputs: ID_Valid

Description:

The ID_Checking_Module verifies whether the entered player ID matches one of the valid user IDs stored in the PlayerID_ROM.hex memory file. The entered ID value is captured using the ID_Load signal and compared against stored player identification entries.

When a valid ID match is detected, the module asserts the ID_Valid signal HIGH and forwards the authentication process to the password verification stage. If the entered ID does not match any valid stored entry, the authentication process is blocked and gameplay access remains disabled.

The module internally performs:

- ID capture and storage
- ROM-based ID lookup
- ID comparison logic
- Authentication control signaling

This module provides secure multiuser identification capability and ensures that only registered users can access the game system.

[illegible]

2.2.3.1 Password_Checking_Module Block Diagram:



Inputs: PasswordInput[3:0], Password_Load, clk, rst

Outputs: Password_Valid, Logged_In, Logged_Out

Description:

The Password_Checking_Module authenticates the player by verifying the entered password against stored password values contained in the Password_ROM.hex memory file. Password digits entered through switches are captured using the Password_Load signal and compared against the stored password sequence associated with the authenticated player ID.

If all password digits match successfully:

- Password_Valid is asserted HIGH
- Logged_In becomes active
- Logged_Out is cleared

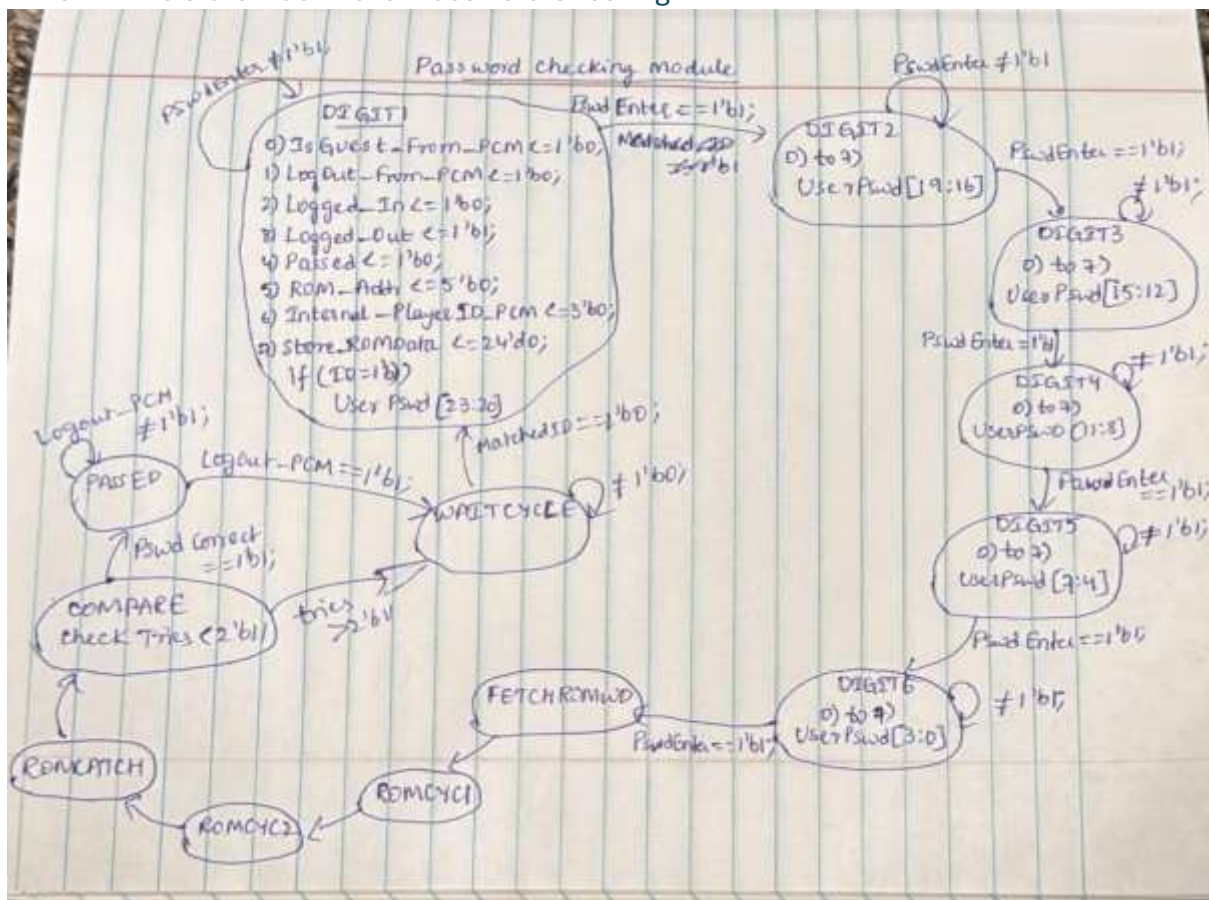
If the password verification fails, the module keeps the system in the logged-out state and disables gameplay access.

The module internally performs:

- Password capture and storage
- Password comparison logic
- ROM-based password lookup
- Login/logout state generation

This module provides secure password authentication and prevents unauthorized access to gameplay operations such as level selection, prime-number generation, score tracking, and timer activation.

2.2.3.1 Finite State Machine for Password Checking:



2.3 ButtonShaper:

2.3.1 ButtonShaper Block Diagram:

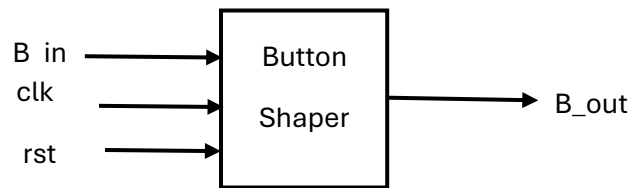


Fig2.3.1: ButtonShaper

Inputs: B_in, clk, rst

Outputs: B_out

Description: The **ButtonShaper** module converts a long button press on **B_in** into a single clock-cycle pulse on **B_out**. When the button is not pressed (**B_in = 1**), the module remains in the **INIT** state and the output **B_out** stays **0**. When the button is pressed (**B_in = 0**), the module transitions to the **PULSE** state and generates exactly one pulse where **B_out = 1** for one clock cycle. After generating this pulse, the module moves to the **WAIT** state, where it ignores the button input while it is still pressed (**B_in = 0**) to prevent repeated pulses. Once the button is released (**B_in = 1**), the module returns to the **INIT** state, making it ready to detect the next button press.

2.3.2 ButtonShaper FSM:

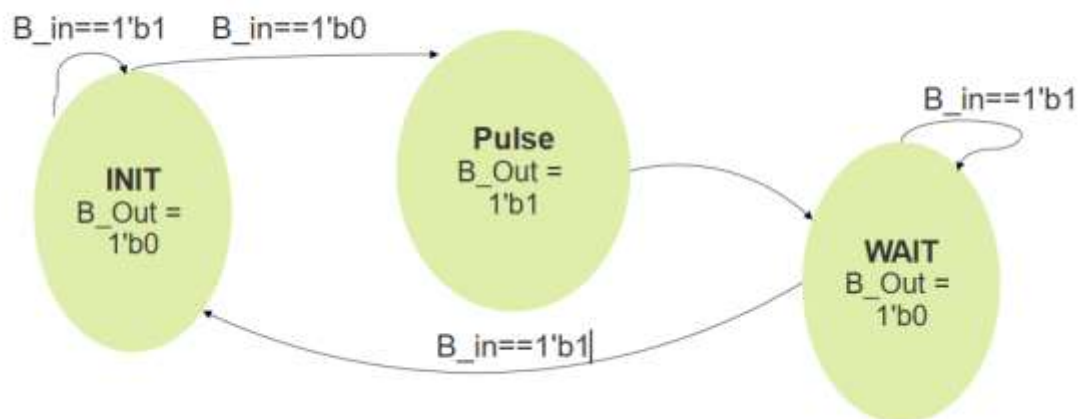


Figure2.3.2: ButtonShaper FSM

2.3.3 ButtonShaper Timing Diagram

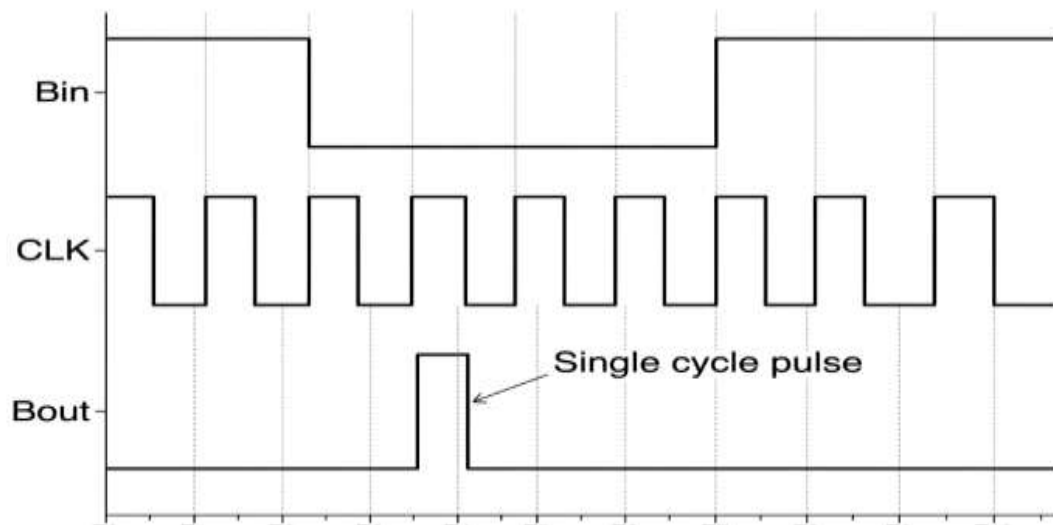


Figure2.5.3: ButtonShaper Timing Diagram

2.4 LevelSelector Module:

2.4.1 LevelSelector Module Block Diagram:

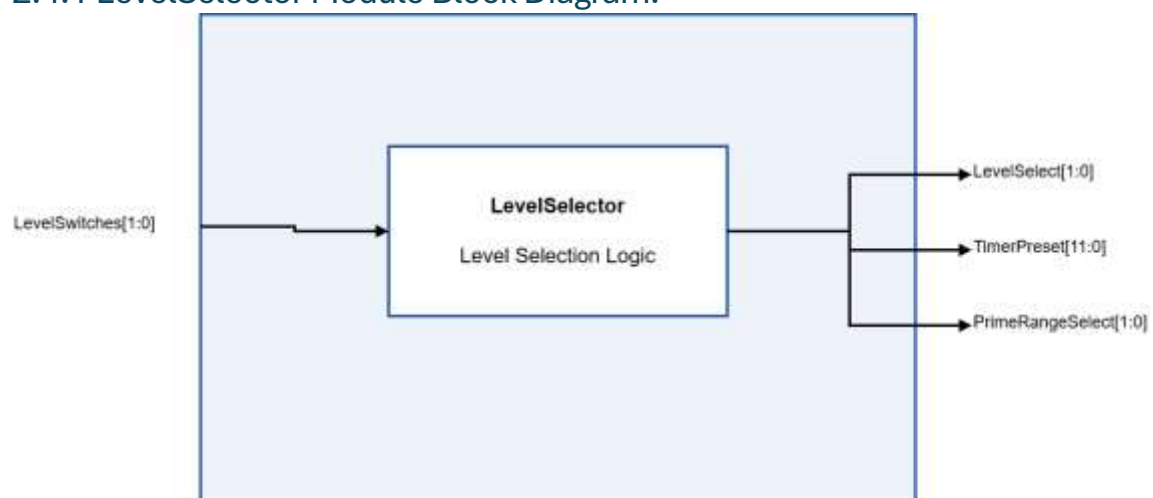


Figure 2.4.1: LevelSelector Block Diagram

Inputs: LevelSwitches[1:0]

Outputs: LevelSelect[1:0], TimerPreset[11:0], PrimeRangeSelect[1:0]

Description:

The LevelSelector module determines the gameplay difficulty level selected by the player. Based on the switch inputs, the module configures:

- Timer duration
- Prime-number range

- Game difficulty parameters

The module supports three gameplay levels:

- Level 1: Small prime-number range with longer timer duration
- Level 2: Medium prime-number range with moderate timer duration
- Level 3: Full prime-number range with shorter timer duration

The output configuration signals are forwarded to the timer module and prime target generator to adjust gameplay behavior dynamically.

2.5 Game Controller:

2.5.1 PrimeGameController Module Block Diagram:

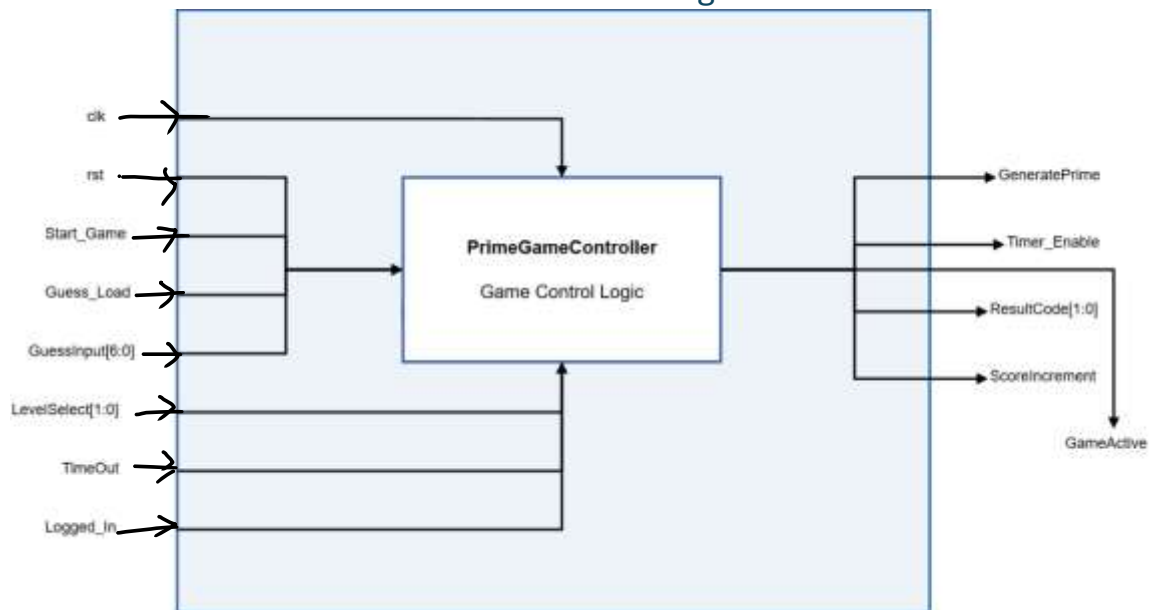


Figure 2.5.1: PrimeGameController Block Diagram

Inputs: clk, rst, Start_Game, Guess_Load, GuessInput[6:0], LevelSelect[1:0], TimeOut, Logged_In, IsPrime

Outputs: GeneratePrime, Timer_Enable, ResultCode[1:0], ScoreIncrement, GameActive

Description:

PrimeGameController module acts as the central control unit of the Prime Number Guessing Game system.

After successful authentication, the controller waits for the player to start the game. Once gameplay begins, the controller enables the timer system and initiates random prime target generation.

During gameplay, the controller coordinates:

- prime target generation
- guess validation
- result generation
- score updates
- round transitions
- timeout handling

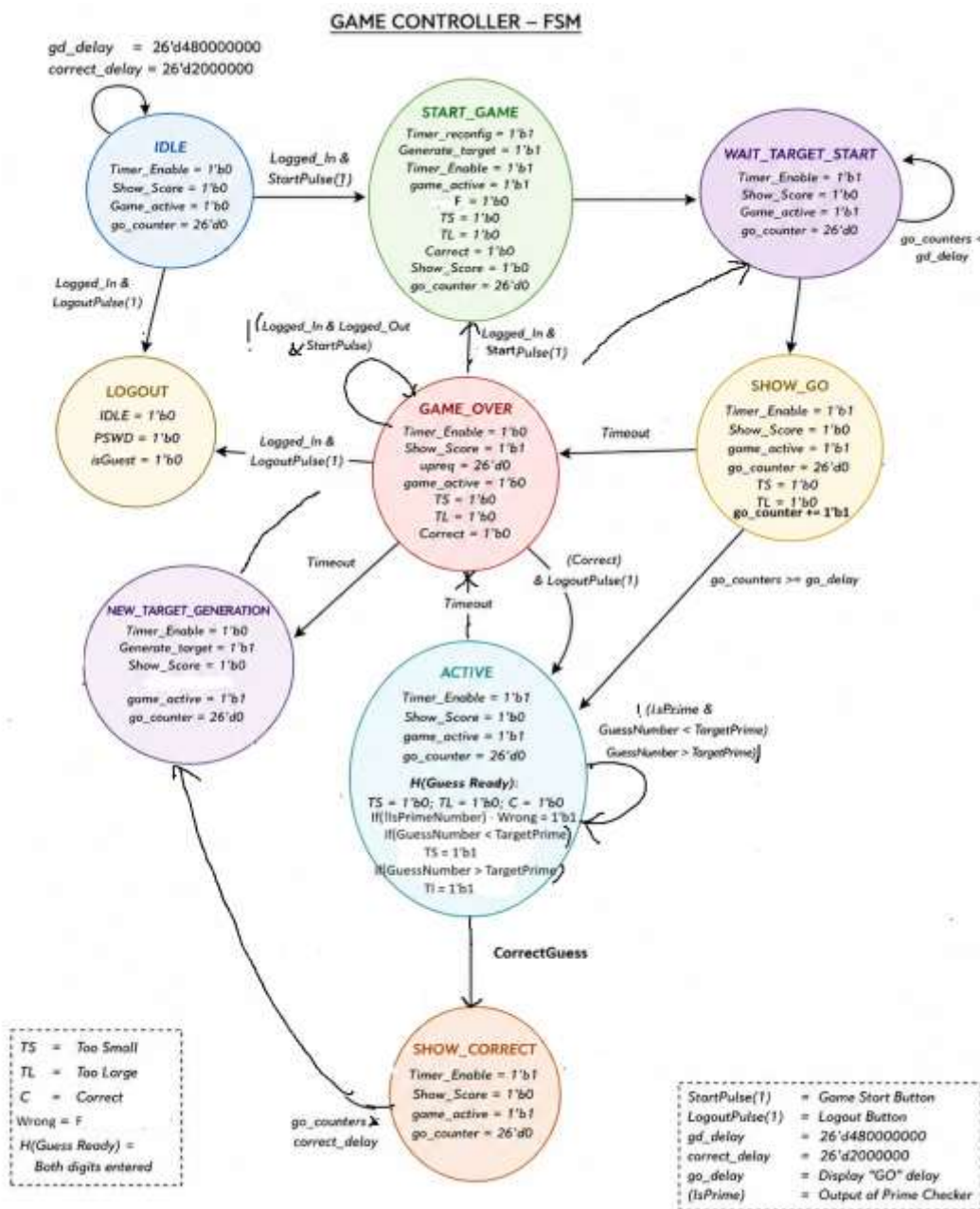
If the entered guess is not prime, the controller generates the WRONG(F) output state. If the guess is prime, the controller compares it with the hidden target prime and generates:

- TOO SMALL (S)
- TOO LARGE (L)
- CORRECT (C)

Whenever the player correctly guesses the hidden target prime, the controller increments the score and generates a new target prime for the next gameplay round.

When the timer expires, gameplay operations are disabled and the final score is displayed.

2.5.2 FSM PrimeGameController Module:



2.6 Prime Generation:

2.6.1 PrimeTargetGenerator Module:

2.6.1.1 PrimeTargetGenerator Module Block Diagram:

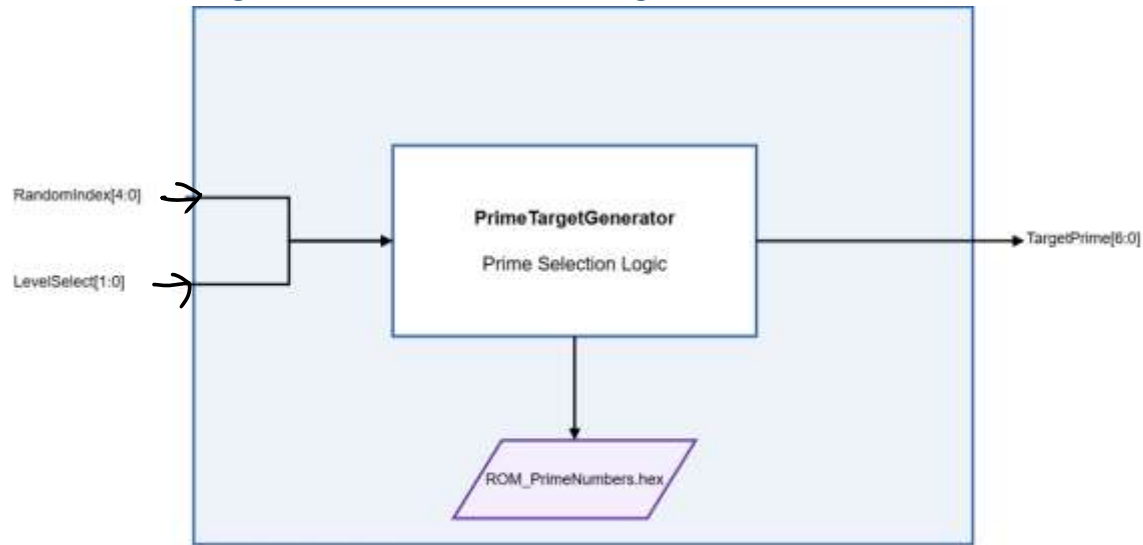


Figure 2.6.1: PrimeTargetGenerator Block Diagram

Inputs: RandomIndex[4:0], LevelSelect[1:0]

Outputs: TargetPrime[6:0]

Description:

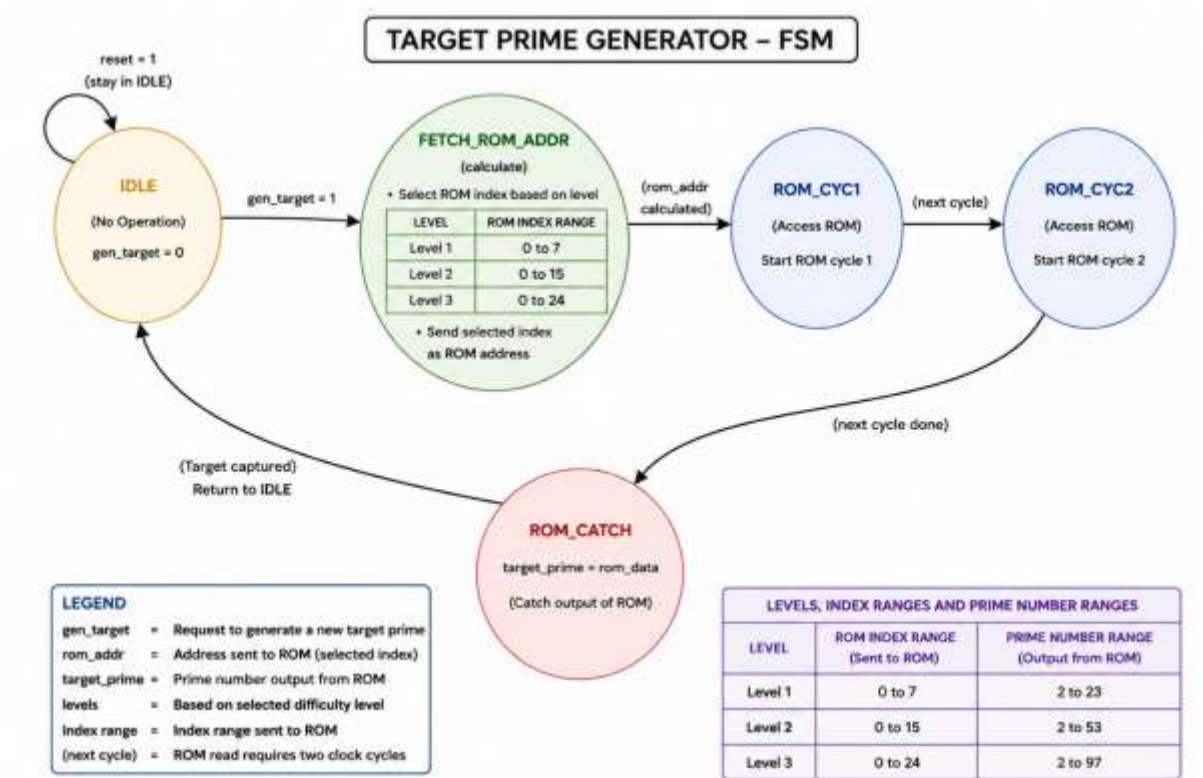
The PrimeTargetGenerator module generates the hidden target prime number used during gameplay. The module receives a pseudo-random index from the RandomIndexLFSR module and uses that index to access a ROM-style lookup table containing valid prime numbers.

Depending on the selected difficulty level, only a subset of the stored prime numbers is accessible:

- Level 1 uses smaller primes
- Level 2 uses medium-range primes
- Level 3 uses the full prime-number set up to 97

Because every entry in the lookup table is guaranteed to be prime, the generated target number is always a valid prime value.

2.6.1.2 FSM for PrimeTargetGenerator Module:



2.6.2 RandomIndexLFSR Module:

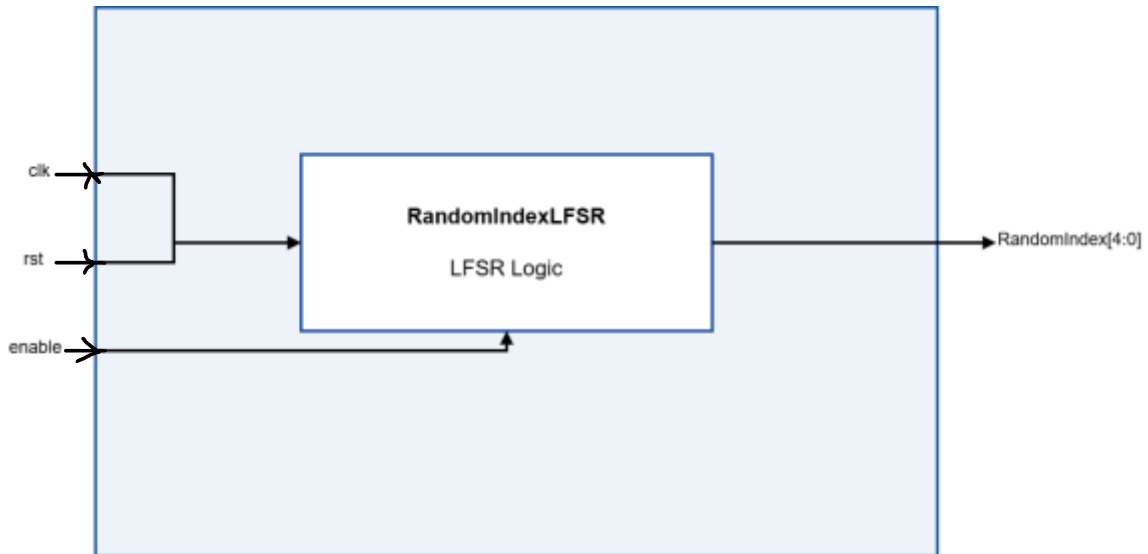


Figure 2.6.2: RandomIndexLFSR Block Diagram

Inputs: clk, rst, enable

Outputs: RandomIndex[4:0]

Description:

The RandomIndexLFSR module implements a Linear Feedback Shift Register (LFSR)-based pseudo-random generator used to create random indices for target prime selection.

When enabled, the LFSR advances through a pseudo-random sequence of states on each rising edge of the clock signal. The generated output value is used as an address index for the PrimeTargetGenerator lookup table.

The use of an LFSR provides hardware-efficient pseudo-random behavior while minimizing FPGA resource usage compared to conventional binary counters.

2.6.3 PrimeLFSR Module:

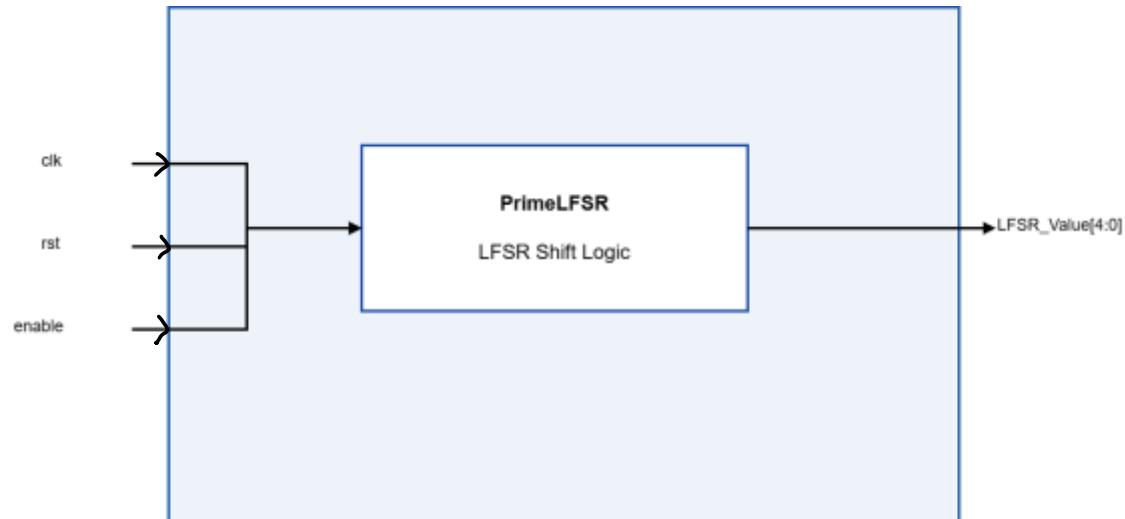


Figure 2.6.3: PrimeLFSR Block Diagram

Inputs: clk, rst, enable

Outputs: LFSR_Value[4:0]

Description:

The PrimeLFSR module implements the internal Linear Feedback Shift Register logic used by the RandomIndexLFSR module.

The LFSR advances through pseudo-random state transitions on each clock cycle and provides the randomness source for target prime-number generation.

2.6.4 ROM_PrimeNumbers Module:

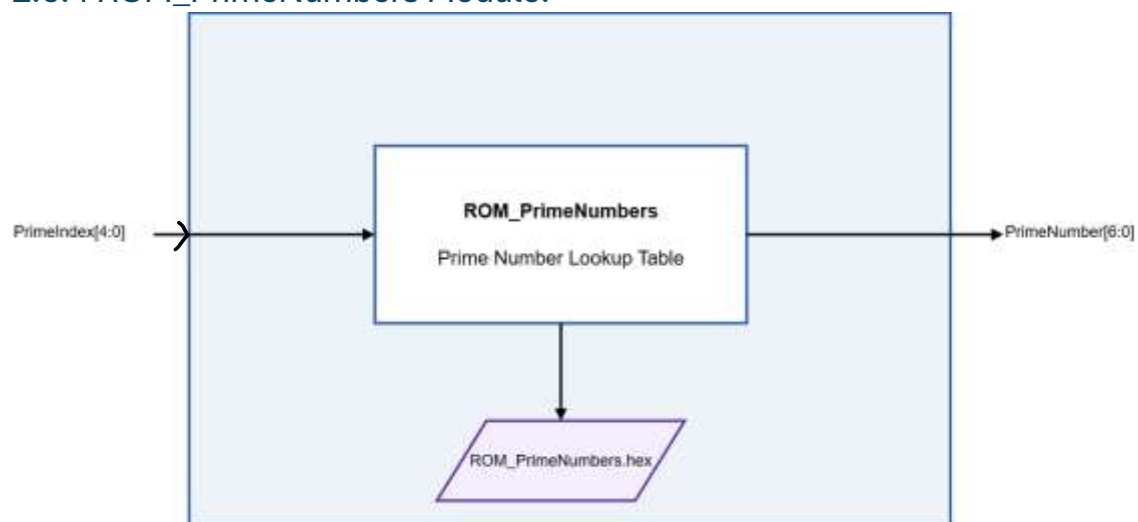


Figure 2.6.4: ROM_PrimeNumbers Block Diagram

Inputs: PrimeIndex[4:0]

Outputs: PrimeNumber[6:0]

Description:

The ROM_PrimeNumbers module stores all valid prime numbers supported by the game system.

The module behaves as a ROM-style lookup table where each address corresponds to a valid prime number less than or equal to 97.

2.7 Guess Input:

2.7.1 TwoDigitGuessInput Module:

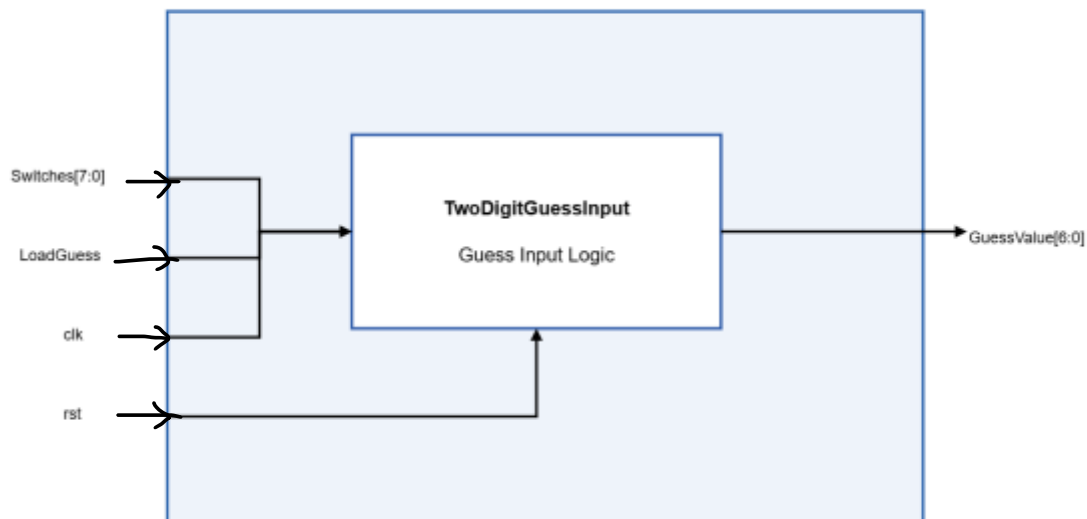


Figure 2.7.1: TwoDigitGuessInput Block Diagram

Inputs: Switches[7:0], LoadGuess, clk, rst

Outputs: GuessValue[6:0]

Description:

The TwoDigitGuessInput module captures and stores the player's entered numerical guess using FPGA switches and push-button controls.

The module supports two-digit decimal input values and provides synchronized storage of the entered guess for gameplay evaluation.

2.8 Prime Checking:

2.8.1 Prime Checking Module:

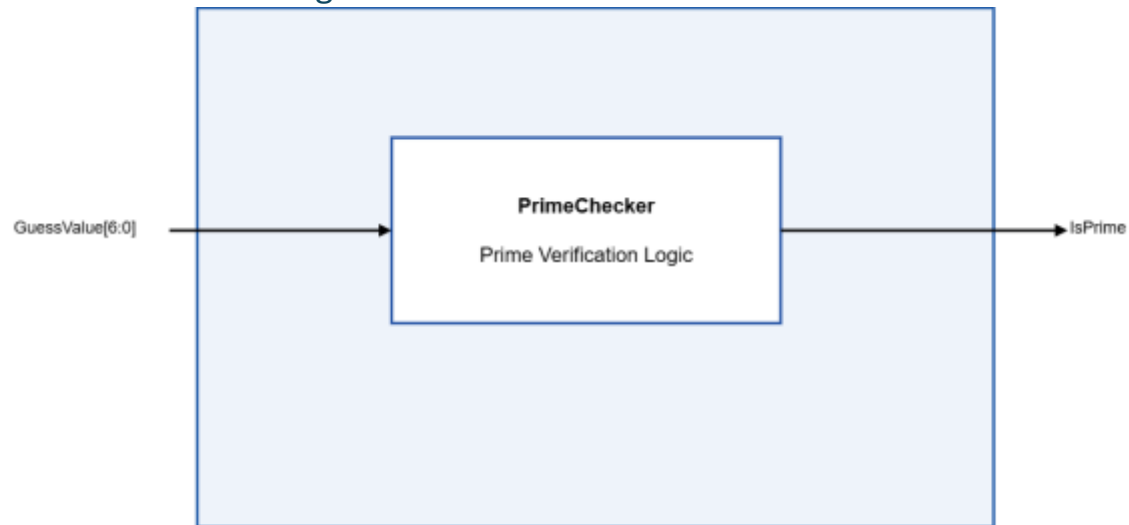


Figure 2.8.1: Prime CheckingBlock Diagram

Inputs: GuessInput[6:0]

Outputs: IsPrime

Description:

The PrimeChecker module determines whether the entered player guess is a valid prime number. Instead of using computational algorithms such as division or modulus operations, the module implements a ROM-style lookup table containing all valid prime numbers less than or equal to 97.

The entered number is compared against the stored prime-number table:

- If a match is found, IsPrime is asserted HIGH
- Otherwise, the output remains LOW

This hardware-oriented lookup approach avoids runtime computation and satisfies FPGA design constraints requiring non-computational prime checking.

2.9 Timer System:

2.9.1 OneSecTimer:

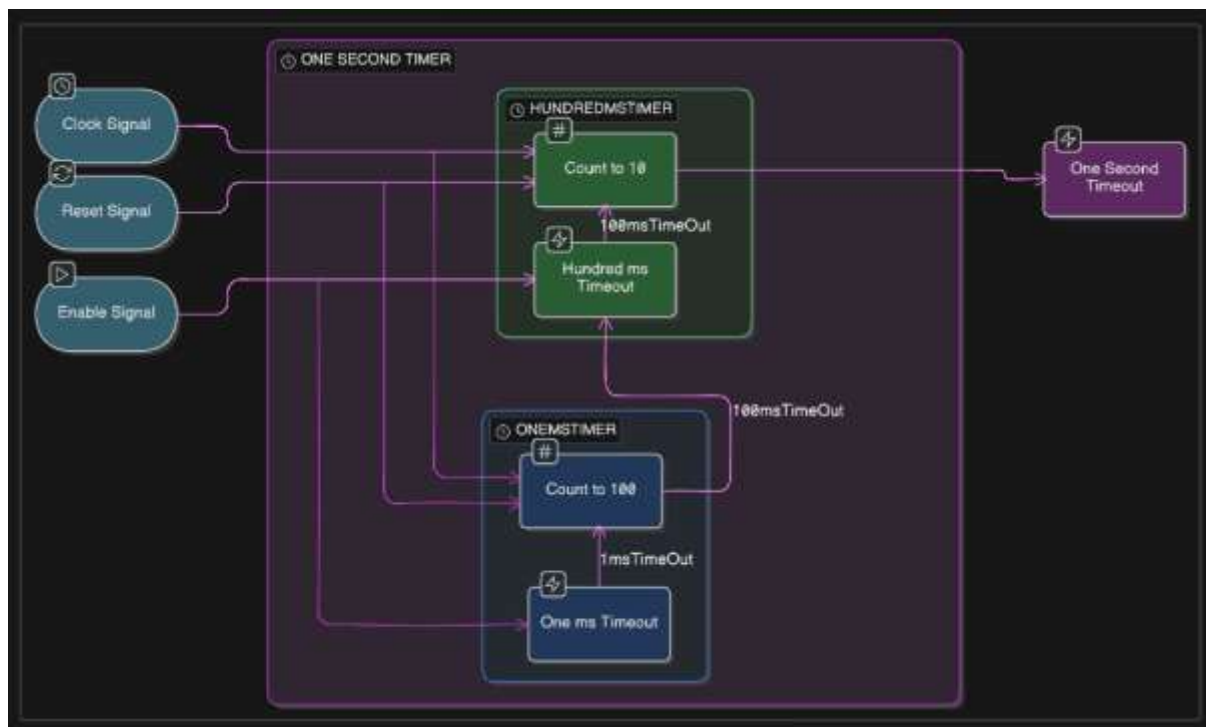


Figure 2.9.1.1: OneSecTimer Block Diagram

Inputs: clk, reset, enable

Outputs: OneSecTimeOut

Description: The nested one-second timer module generates a one-second timeout pulse using a hierarchical structure of smaller timer modules. It receives the clock, reset, and enable signals as inputs. The design operates in multiple stages, where a lower-level timer first generates a shorter timeout pulse, and this pulse is then used as the input enable for the next stage. A 1 ms timer first produces a 1 ms timeout pulse by counting a fixed number of clock cycles. This output is passed to a 100 ms timer, which counts 100 such pulses to generate a 100 ms timeout signal. Finally, another counter counts 10 pulses of the 100 ms timeout signal to produce the one-second timeout output. When enable is HIGH, the timer runs normally and generates the one-second pulse. When reset is asserted, all counters are cleared and the timing sequence restarts from the beginning.

Note: In Lab4, we replaced 1ms counter module with 1ms LFSR module

2.9.1.2 OnemillisecLFSRTimer Module:

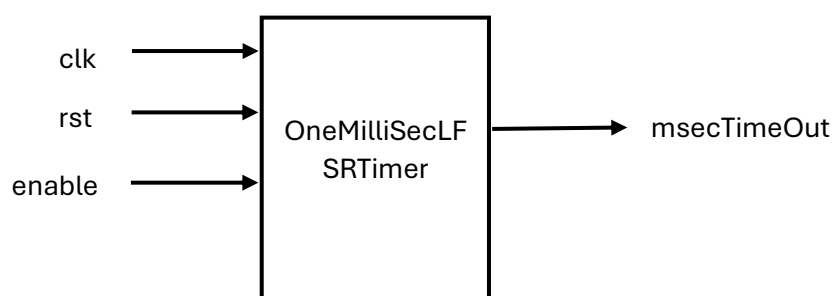


Figure 2.9.1.2: OneMilliSecLFSRTimer Block Diagram

Inputs: clk, rst, enable

Output: msecTimeOut

Description: The diagram shows the architecture of an LFSR-based 1 ms timer module. The design receives three input signals: clock, reset, and enable. Unlike the regular 1 ms timer from Lab 3, which generates the timeout pulse by counting clock cycles in normal binary order, this module implements a Linear Feedback Shift Register (LFSR)-based timer. When enabled, the LFSR advances through a pseudo-random sequence of states on each rising edge of the clock. A specific terminal LFSR state is used to determine when the 1 ms interval has elapsed. When that terminal state is reached, the module generates a single-clock-cycle msecTimeOut pulse and restarts the LFSR sequence from its initial seed value. In this way, the module behaves like the Lab 3 1 ms timer in terms of timing, but uses LFSR state transitions instead of a conventional binary counter.

2.9.2 TwoDigitTimer:

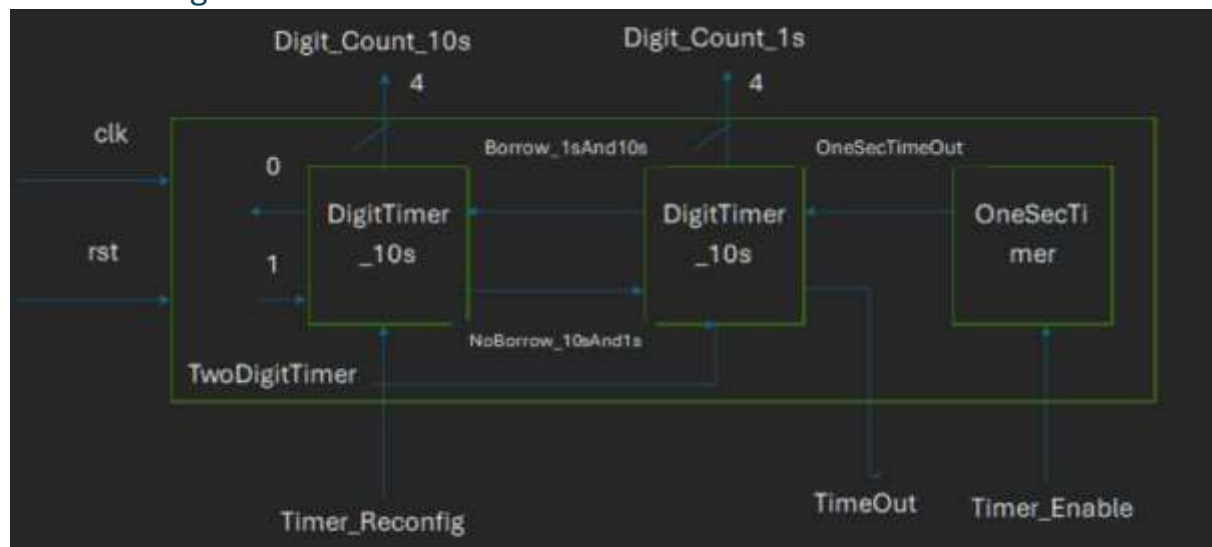


Figure 2.9.2: TwoDigitTimer Block Diagram

Inputs: clk, rst, enable, timer_reconfig

Outputs: TensDigit[3:0], OnesDigit[3:0], TimeOut

Description: The Two Digit Timer module implements a countdown timer using the OneSecondTimer module as its timing source. It receives clk, rst, enable, and timer_reconfig as inputs. The OneSecondTimer generates a one-second timeout pulse, which is used to decrement the ones digit counter. When the ones digit rolls over during countdown, a borrow signal is generated to decrement the tens digit counter. Together, the two counters form a two-digit decimal timer that can count down from an initial value such as 99 to 00. The timer_reconfig signal reloads the timer to its starting value, enable allows the timer to run, and TimeOut is asserted when both digits reach zero.

2.9.3 ThreeDigitTimer Module:

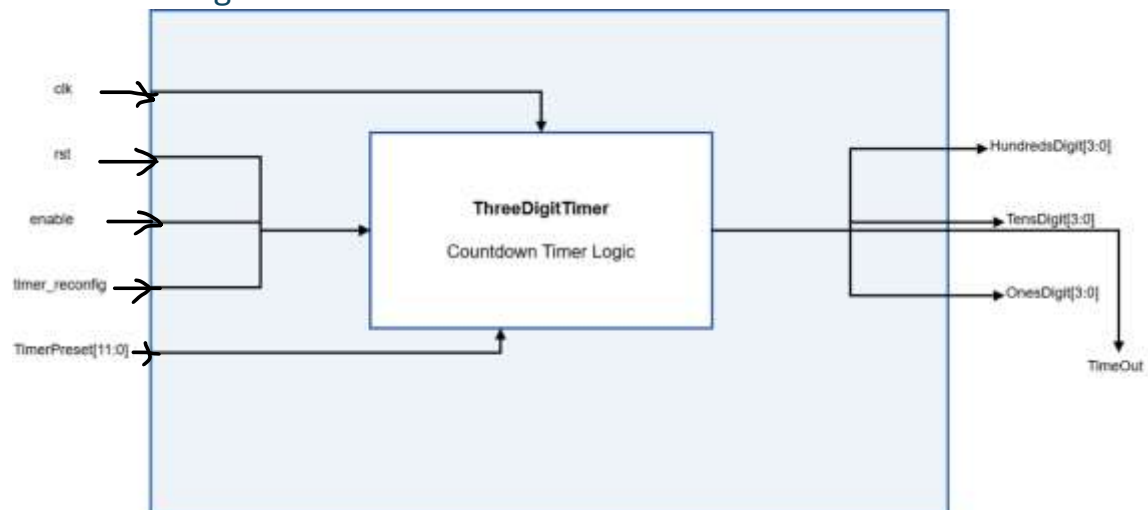


Figure 2.9.3: ThreeDigitTimer Block Diagram

Inputs: clk, rst, enable, timer_reconfig, TimerPreset[11:0]

Outputs: HundredsDigit[3:0], TensDigit[3:0], OnesDigit[3:0], TimeOut

Description:

The ThreeDigitTimer module implements a configurable three-digit countdown timer used to control gameplay duration. The timer receives preset values from the LevelSelector module depending on the selected difficulty level.

The timer decrements once every second using the OneSecondTimer module as its timing source. The ones digit decrements first, followed by borrow propagation to the tens and hundreds digits when required.

When the timer reaches 000, the module asserts the TimeOut signal, which disables gameplay operations and triggers final score display.

The timer supports:

- Reset initialization
- Dynamic timer reload
- Start/stop control
- Difficulty-based timer configuration

2.10 ScoreBoard:

2.10.1 ScoreBoard Module Block Diagram:

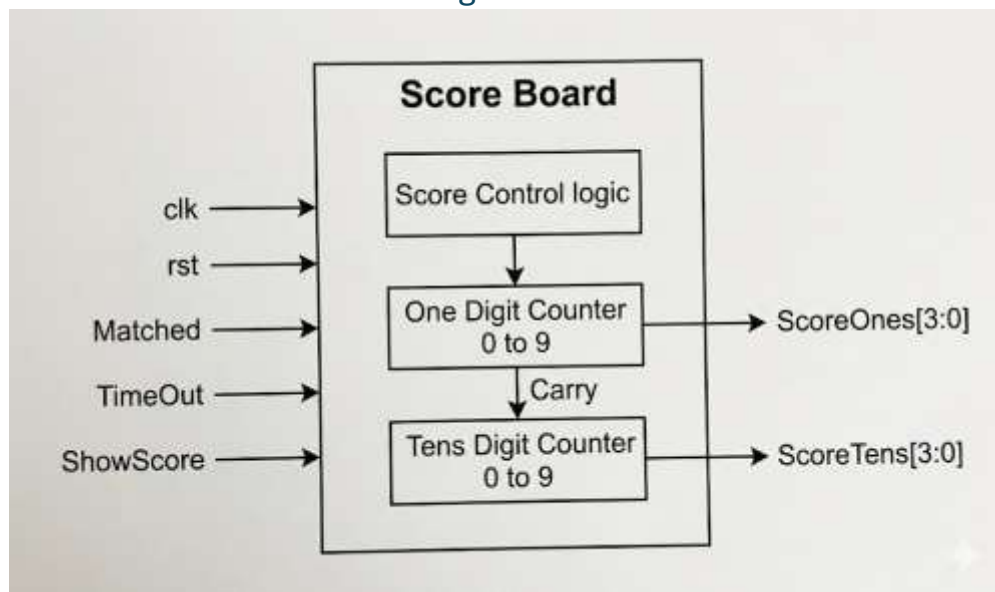


Figure 2.10.1: ScoreBoard Block Diagram

Inputs: clk, rst, Match, TimeOut, ShowScore

Outputs: ScoreTens[3:0], ScoreOnes[3:0]

Description: The ScoreBoard module counts the number of successful rounds completed by the player during gameplay and stores the result as a 2-digit decimal score. It receives clk, rst, Match, TimeOut, and ShowScore as inputs. Whenever a valid match occurs, the module increments the score by one. The ones digit counter counts from 0 to 9, and when it overflows, it generates a carry to increment the tens digit counter. This allows the module to keep track of scores from 00 to 99. The rst signal clears the score counters, TimeOut indicates the end of the game, and ShowScore is used to control when the score should be displayed on the seven-segment displays.

2.11 Display System:

2.11.1 DisplayController:

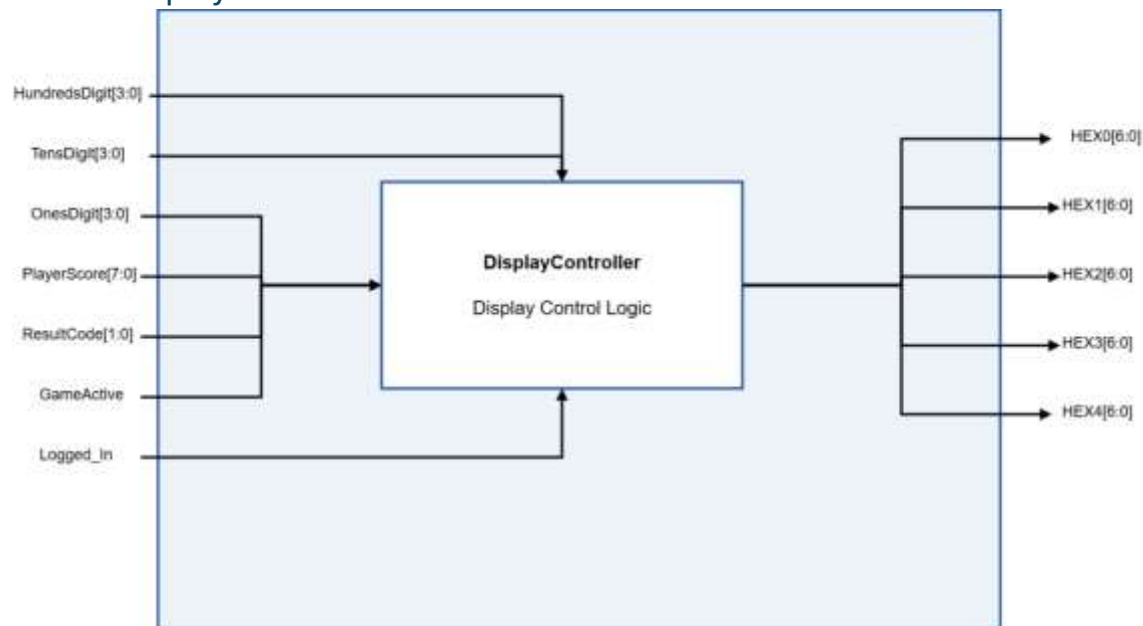


Figure 2.11.1: DisplayController Block Diagram

Inputs: clk, rst, GameState, ResultCode[1:0], TimerDigits[11:0], ScoreDigits[7:0], GuessValue[6:0]

Outputs: HEX0–HEX5[6:0]

Description:

The DisplayController module manages all seven-segment display outputs used throughout the game. The module dynamically selects which information should appear on the displays depending on the current game state.

During authentication, the displays show login-related information. After successful login, the module displays the countdown timer and gameplay messages. During gameplay, the module displays feedback messages such as:

- GO
- FAIL
- TOO SMALL
- TOO LARGE
- CORRECT

The DisplayController also controls the scoreboard display after timeout and manages multiplexing between timer digits, result messages, and score values. This module centralizes all display-related operations and simplifies top-level system integration.

2.11.2 7SegmentDecoder:

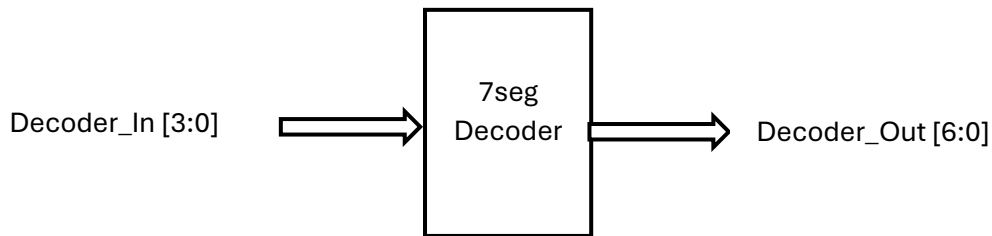


Fig2.11.2: My7segmentDecoder_4To7

Inputs: Decoder_In [3:0]

Outputs: Decoder_Out [6:0]

Module Description: The 7-segment decoder module converts a 4-bit binary input (Decoder_In) into the corresponding 7-segment display pattern (Decoder_Out). Based on the input value (0 to F), the module activates specific segments (a through g) of the display to represent the appropriate hexadecimal digit. Each bit of Decoder_Out controls one segment and determines whether that segment is ON or OFF.

2.12 LoadRegister

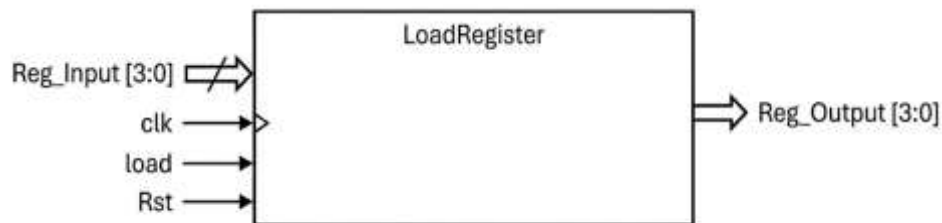


Figure: LoadRegister Block Diagram

Figure2.12: LoadRegister Block Diagram

Inputs: Reg_Input [3:0], clk, Rst, load

Outputs: Reg_Output [3:0]

Description: The Load Register module stores a 4-bit input value (Reg_Input) into the register output (Reg_Output) when the load signal is asserted. The register updates on the rising edge of the clock (clk). When the reset signal (Rst) is LOW, the output is cleared to 0000. The module ensures that the input value is loaded only when the load signal is enabled.

2.13 Score Tracking and Winner Detection

2.13.1 Score Tracking and Winner Detection Block Diagram

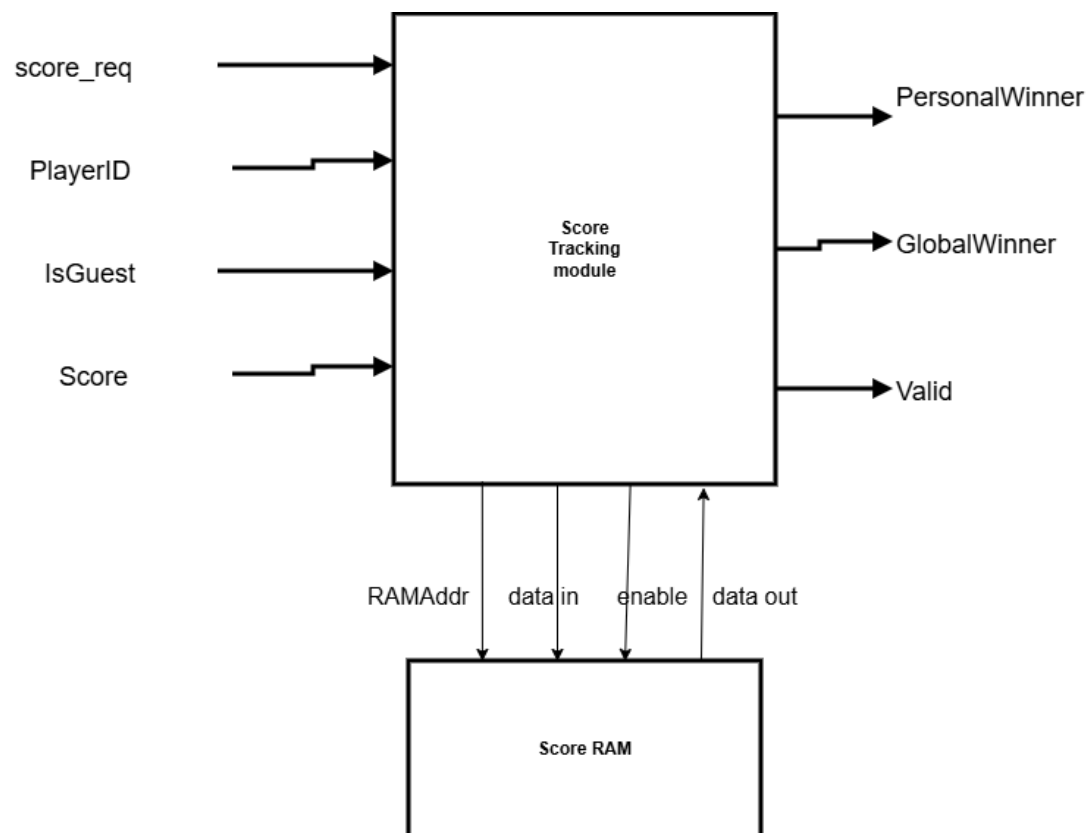


Figure2.13: Score Tracking and Winner Detection

Inputs: `score_req`, `PlayerID`, `IsGuest`, `Score`.

Outputs: `PersonalWinner`, `GlobalWinner`, `Valid`.

RAM Interface Signals: `RAMAddr`, `data in`, `enable`, `data out`.

Description: The ScoreTracking module manages player score storage, personal best score comparison, and global winner detection for the Prime Number Guessing Game. The module interfaces with the Score RAM module to store and retrieve player scores using the player identification value provided by the game controller.

When gameplay is completed, the `score_req` signal is asserted along with the player ID and the final score achieved during the game. The ScoreTracking module accesses the Score RAM using the `RAMAddr` signal to retrieve the previously stored score associated with the player.

The module compares the current score with the previously stored score:

- If the new score is greater than the stored score, the module updates the Score RAM through the `data in` and `enable` signals and asserts the `PersonalWinner` signal.
- If the score does not exceed the stored value, the previous score is preserved.

The module also maintains a global winner record by continuously tracking the highest score achieved among all players. If the current score exceeds the existing global best score, the `GlobalWinner` signal is asserted.

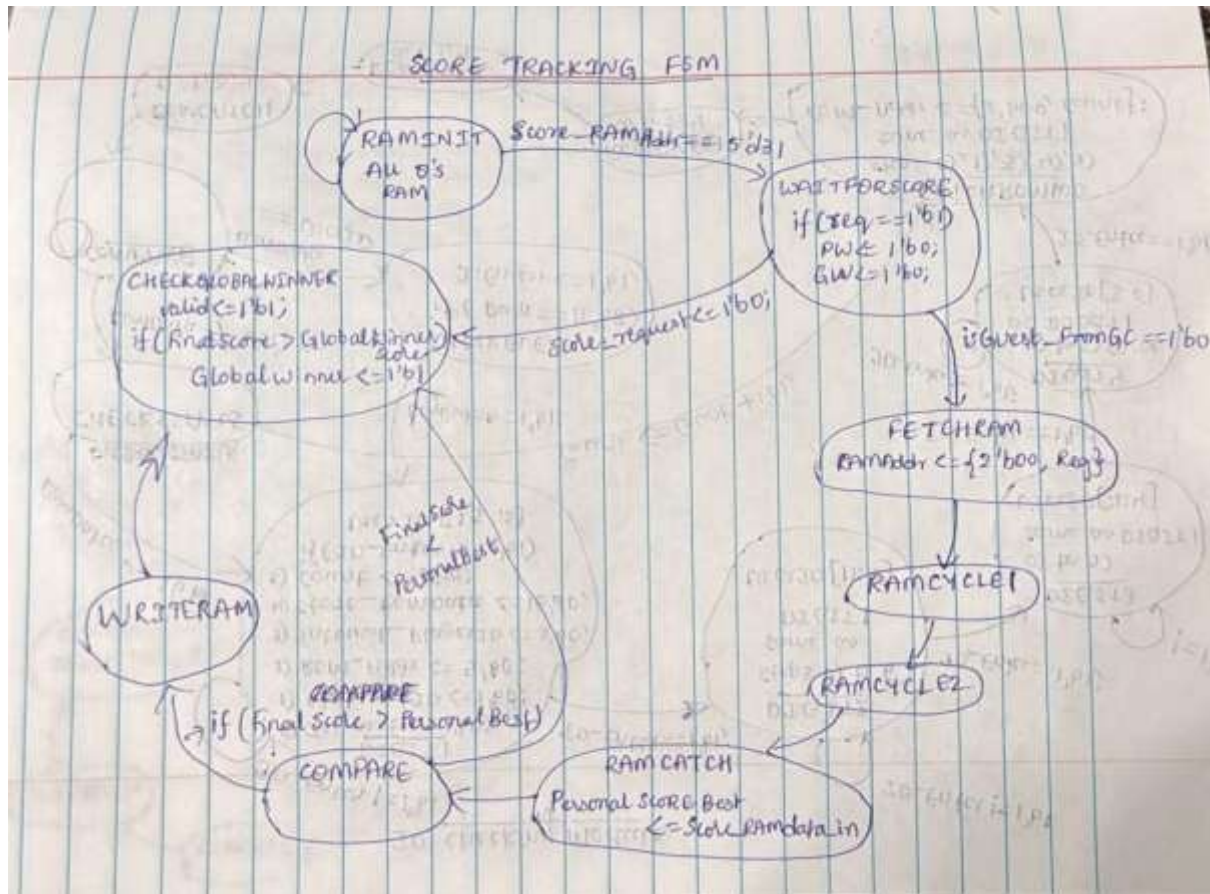
The Valid signal indicates that the score comparison and RAM update operations have completed successfully.

The ScoreTracking module communicates with the Score RAM using:

- RAMAddr for memory addressing
- data in for RAM write operations
- enable for memory write control
- data out for retrieving stored score values

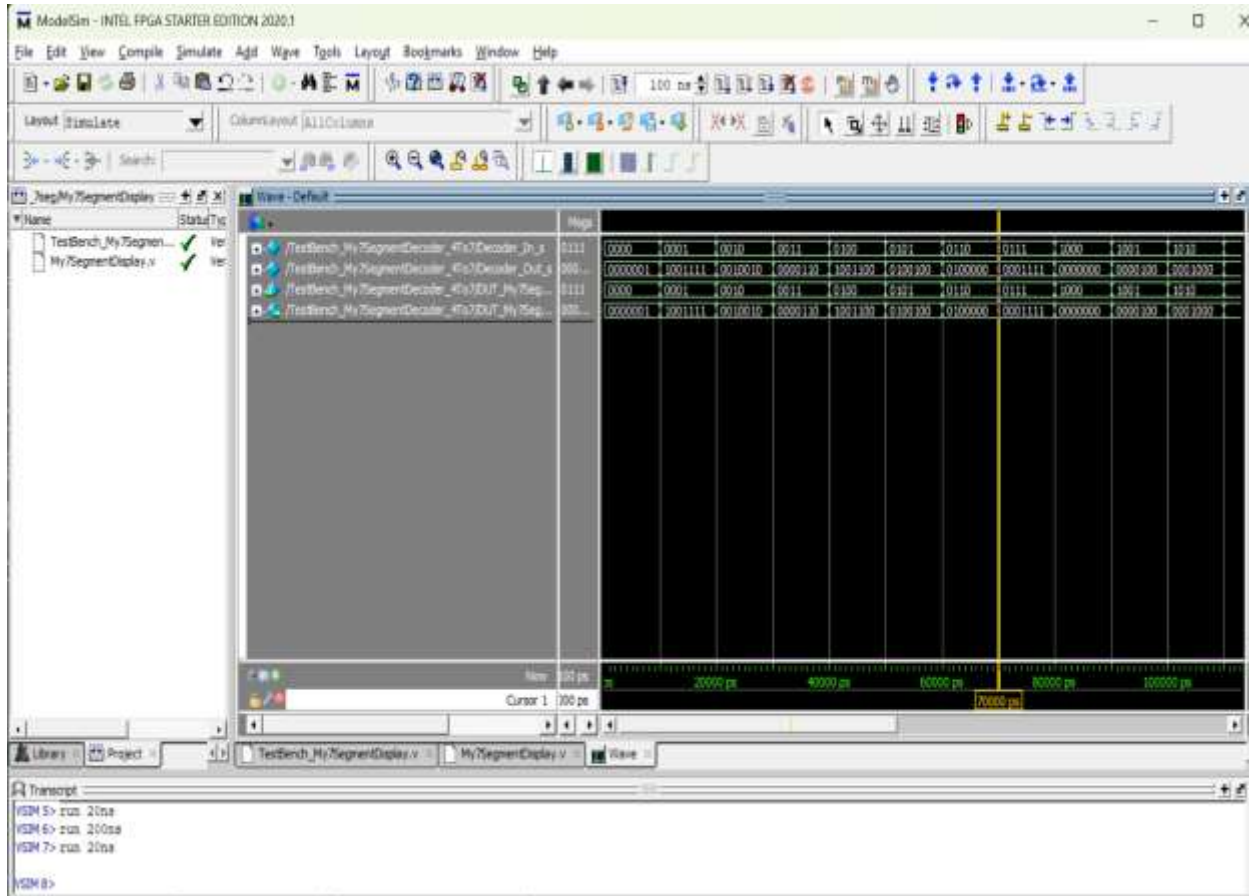
This module enables persistent score management and winner detection functionality within the game system.

2.13.2 Finite State Machine for Score Tracking Module:



3 Simulation Results:

3.1 7-Segment Decoder (My7SegmentDecoder_4to7) Simulation:

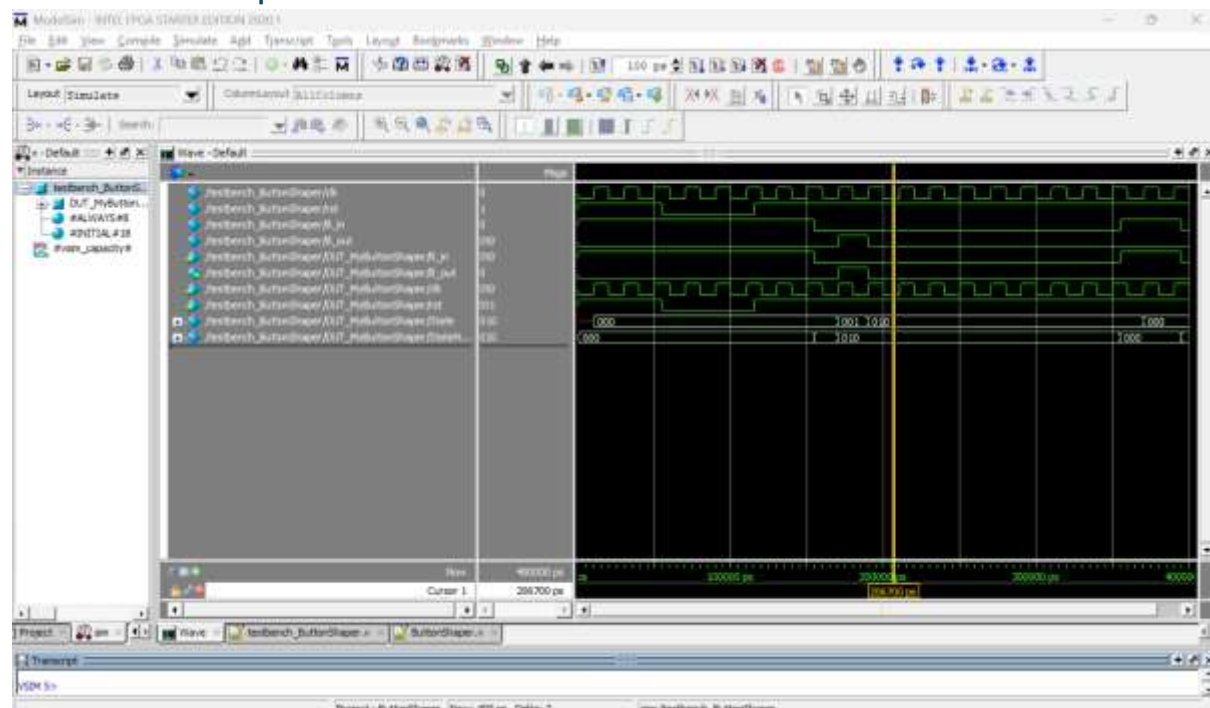




The simulation shown in the screenshots verifies the functionality of the 4-bit to 7-segment decoder module. The input Decoder_In is driven through all possible 4-bit combinations from 0000 to 1111, and the second below immediate bus is the output Decoder_Out updates immediately for each change in the input bus. The waveform shows 7 segment activation as per the 7segmentDisplay behaviour for example ON-0 and for OFF-1 for each hexadecimal input value.

Critical Testing: Decoder_In = 0000, which accurately displays the number 0, and Decoder_In = 1000, in which all seven segments are engaged to display the maximum segment utilization of 8, are critical test cases. Correct hexadecimal display patterns are also produced by inputs ranging from 1010 to 1111 (A to F). These findings validate correct decoding for every possible combination of inputs.

3.2 ButtonShaper Simulation:



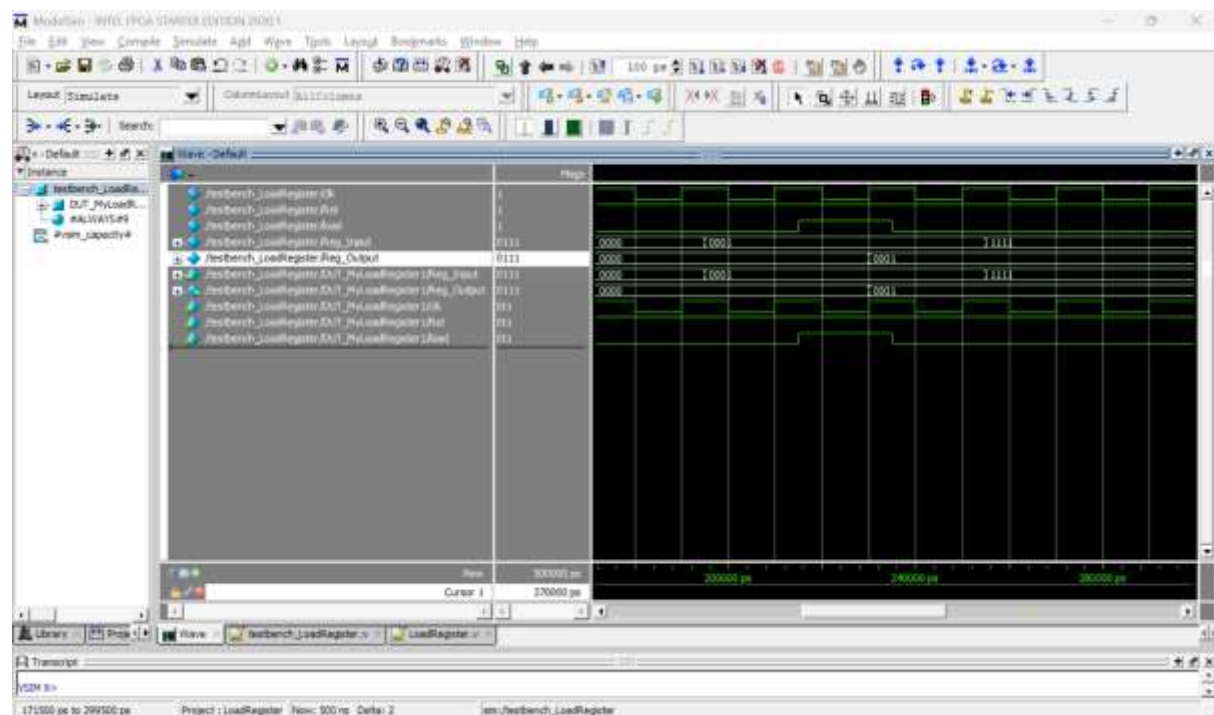
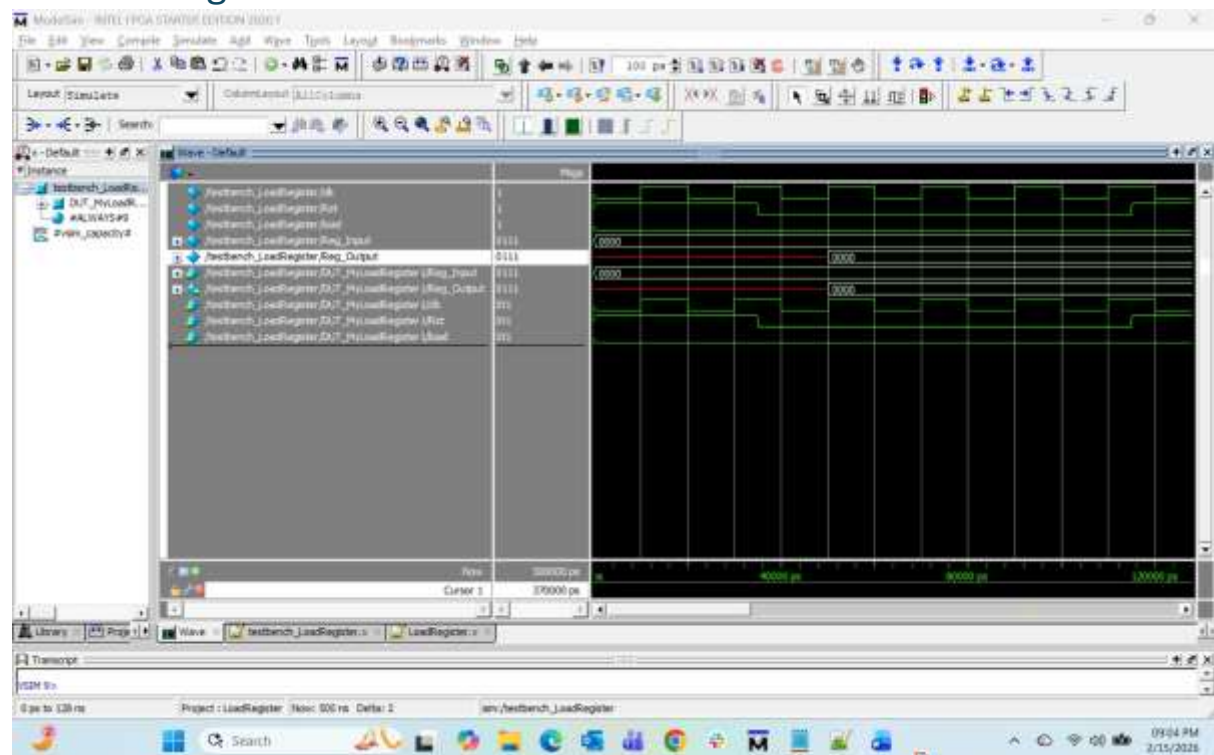
Button Shaper Simulation Results

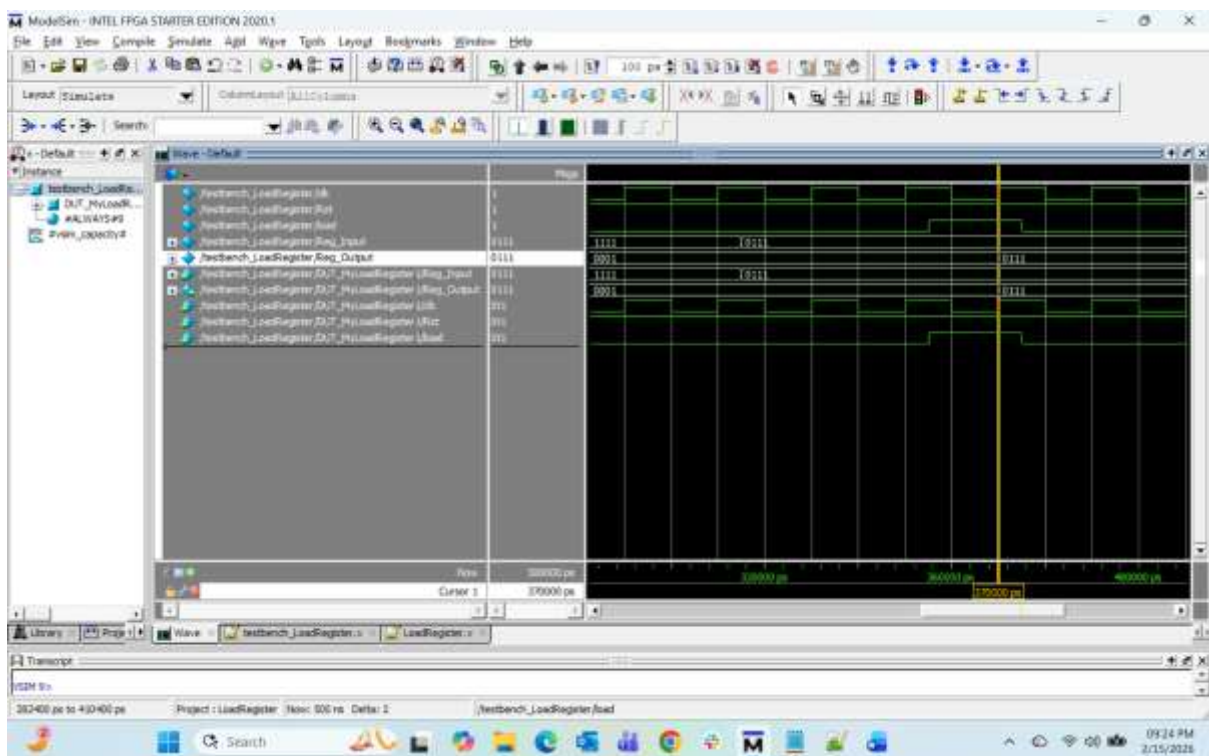
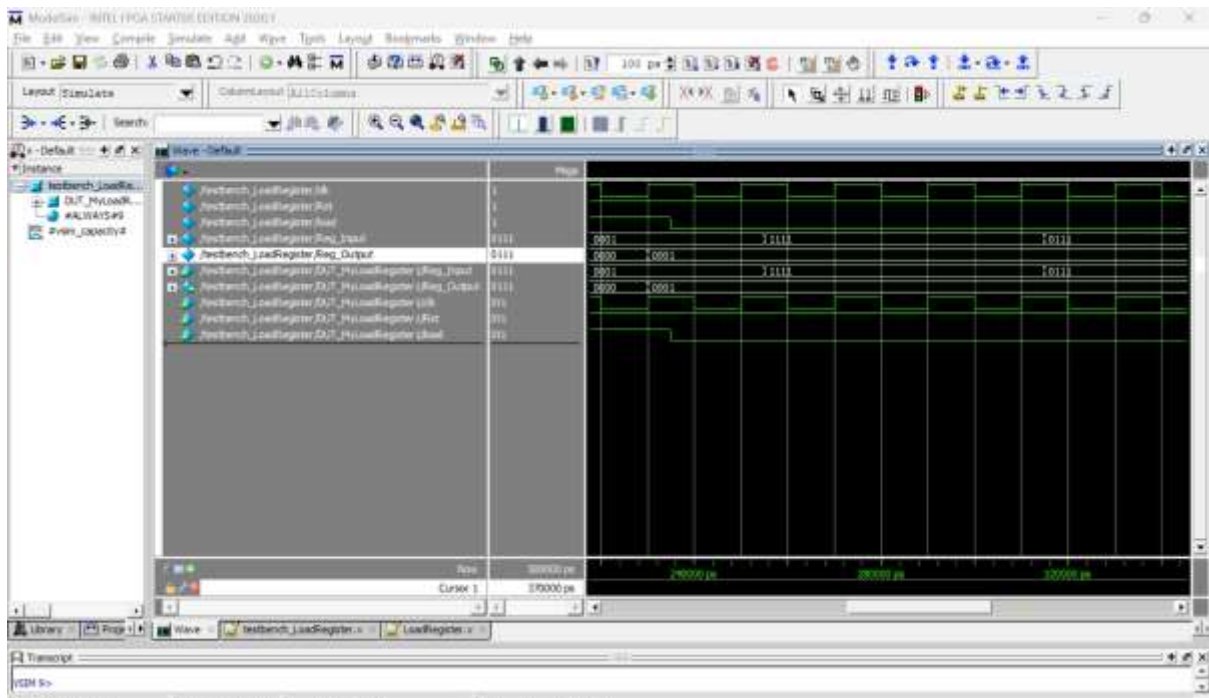
The simulation results for the Button Shaper module are shown in the waveform. To verify the functionality, the input button signal (B_in) is varied to represent different button press conditions, including long pulses and rapid transitions. The waveform shows that the module converts a long or unstable input pulse into a single-cycle output pulse (B_out) synchronized with the clock signal (clk). This confirms that the Button Shaper removes unwanted multiple transitions caused by mechanical button behavior and generates a clean control signal.

The simulation also verifies the state transitions of the finite state machine inside the Button Shaper. When the input button is pressed, the module detects the transition and produces a single HIGH pulse at the output, regardless of how long the input remains HIGH. After generating the pulse, the output returns to LOW while the system waits for the next valid button press.

Critical Testing: The simulation verifies that when B_in remains HIGH for multiple clock cycles, the output B_out is asserted for only one clock cycle, demonstrating proper pulse shaping behavior. The reset signal (rst) is also tested to confirm that the module returns to the initial state when reset is activated. These results confirm correct operation of the Button Shaper, reliable edge detection, and proper synchronization with the clock signal.

3.3 LoadRegister Simulation:





Load Register Simulation Results

The simulation results for the Load Register module are shown in the waveform screenshots. To verify correct operation, the input signal Reg_Inp [3:0] is varied across different binary values while controlling the load, clk, and Rst signals. The waveform demonstrates that the output Reg_Out [3:0] updates only on the rising edge of the clock (clk) when the load signal is asserted HIGH.

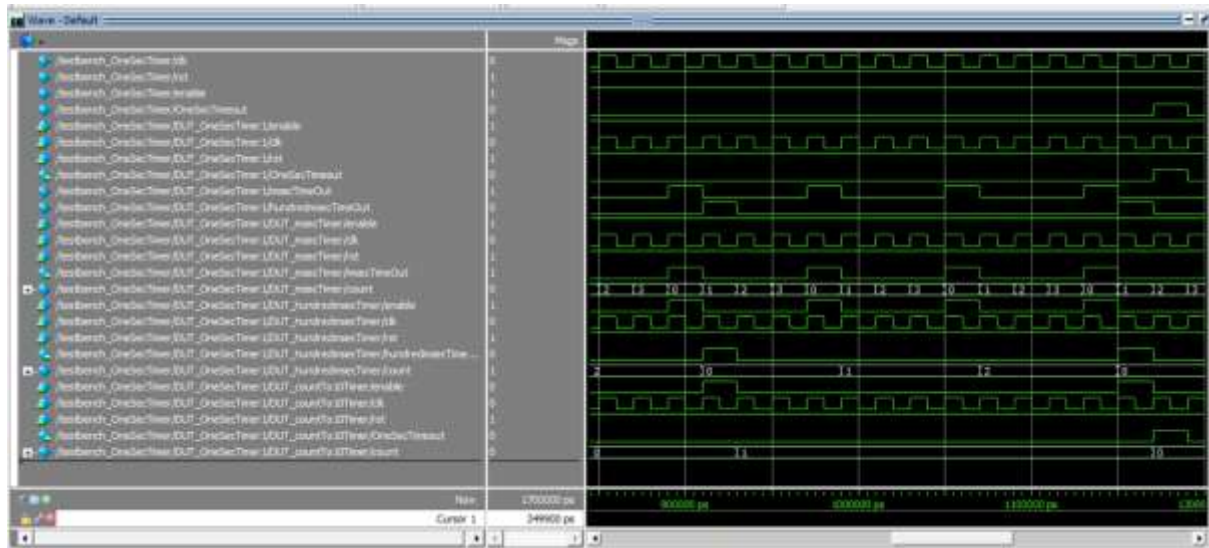
The simulation confirms that when load = 0, changes to Reg_Inp do not affect Reg_Out, indicating that intermediate switch changes are properly ignored. When load = 1 and a clock

edge occurs, the register captures and stores the input value. The stored value remains stable until the next valid load operation.

Critical Testing: Reset behavior is verified by asserting $Rst = 0$, which immediately clears Reg_Output to 0000, confirming proper reset functionality. Load-enable conditions are tested by toggling the load signal while varying the input values (e.g., 0001, 0111, 1111). The output updates only when load is HIGH during a clock edge, confirming correct synchronous storage behavior. These results validate proper clocked operation, controlled loading, and reliable reset handling of the Load Register module.

3.4 OneSecTimer:

Wave - Default		Msgs
/testbench_OneSecTimer/dk	0	
/testbench_OneSecTimer/rst	1	
/testbench_OneSecTimer/enable	1	
/testbench_OneSecTimer/OneSecTimeout	1	
/testbench_OneSecTimer/DUT_OneSecTimer1/enable	1	
/testbench_OneSecTimer/DUT_OneSecTimer1/dk	0	
/testbench_OneSecTimer/DUT_OneSecTimer1/rst	1	
/testbench_OneSecTimer/DUT_OneSecTimer1/OneSecTimeout	1	
/testbench_OneSecTimer/DUT_OneSecTimer1/msecTimeOut	0	
/testbench_OneSecTimer/DUT_OneSecTimer1/hundredmsecTimeOut	0	
/testbench_OneSecTimer/DUT_OneSecTimer1/DUT_msecTimer/enable	1	
/testbench_OneSecTimer/DUT_OneSecTimer1/DUT_msecTimer/dk	0	
/testbench_OneSecTimer/DUT_OneSecTimer1/DUT_msecTimer/rst	1	
/testbench_OneSecTimer/DUT_OneSecTimer1/DUT_msecTimer/msecTimeOut	0	
/testbench_OneSecTimer/DUT_OneSecTimer1/DUT_msecTimer/count	2	
/testbench_OneSecTimer/DUT_OneSecTimer1/DUT_hundredmsecTimer/enable	0	
/testbench_OneSecTimer/DUT_OneSecTimer1/DUT_hundredmsecTimer/dk	0	
/testbench_OneSecTimer/DUT_OneSecTimer1/DUT_hundredmsecTimer/rst	1	
/testbench_OneSecTimer/DUT_OneSecTimer1/DUT_hundredmsecTimer/hundredmsecTime...	0	
/testbench_OneSecTimer/DUT_OneSecTimer1/DUT_hundredmsecTimer/count	0	
/testbench_OneSecTimer/DUT_OneSecTimer1/DUT_countTo10Timer/enable	0	
/testbench_OneSecTimer/DUT_OneSecTimer1/DUT_countTo10Timer/dk	0	
/testbench_OneSecTimer/DUT_OneSecTimer1/DUT_countTo10Timer/rst	1	
/testbench_OneSecTimer/DUT_OneSecTimer1/DUT_countTo10Timer/OneSecTimeout	1	
/testbench_OneSecTimer/DUT_OneSecTimer1/DUT_countTo10Timer/count	0	
Now		1700000 ps
Cursor 1		703300 ps



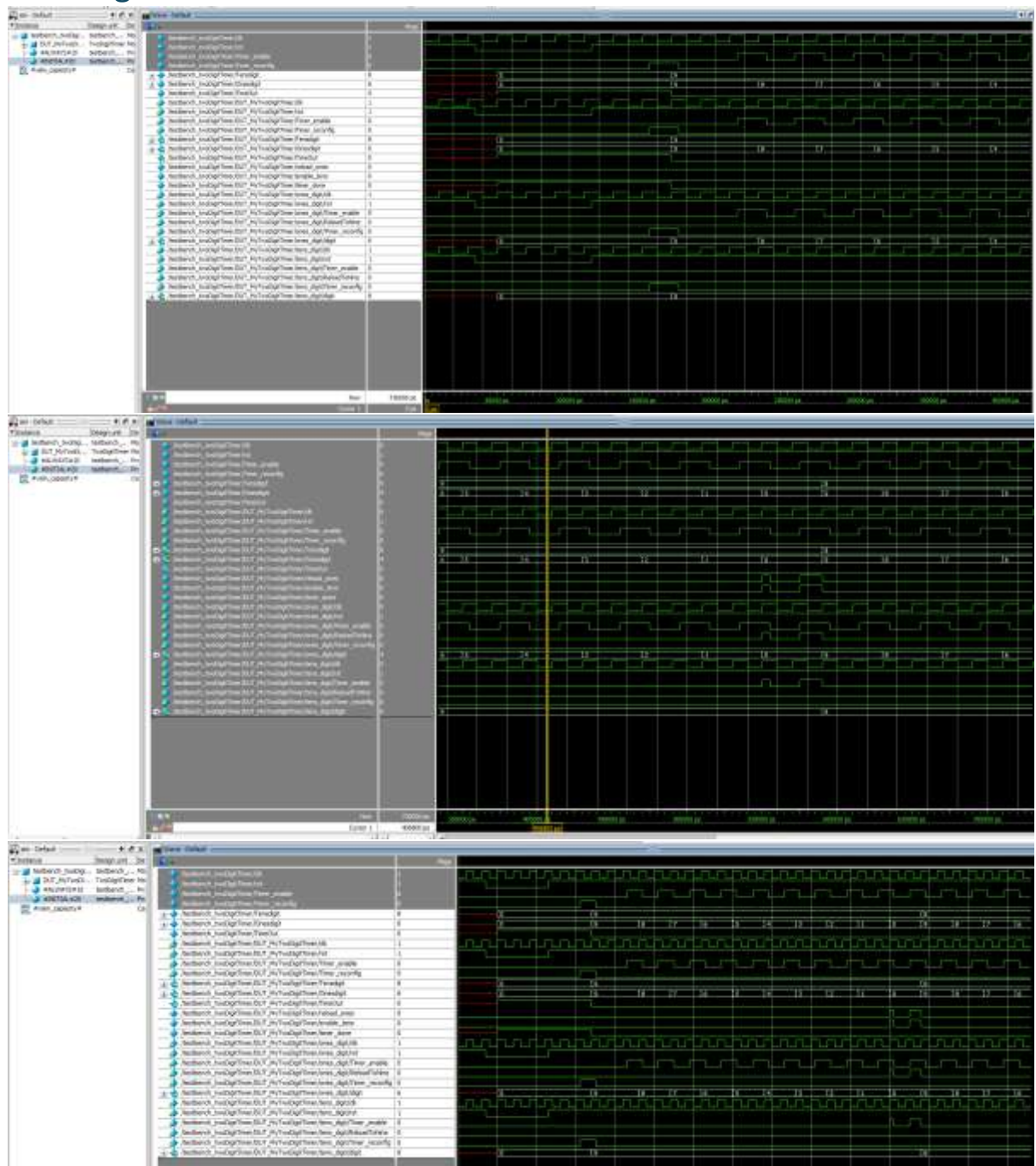
OneSecTimer Simulation Results

The simulation results for the OneSecTimer module are shown in the waveform screenshots. To verify correct operation, the input signals `clk`, `rst`, and `enable` are applied while observing the intermediate timeout signals and the final `OneSecTimeout` output. The waveform demonstrates that the timer operates as a nested structure, where the lower-level timer stages generate intermediate timeout pulses that are used to drive the next stage.

The simulation confirms that at the beginning, when `rst` is asserted, all internal counters are cleared to zero. After reset is released and `enable` is set to HIGH, the timer begins counting clock cycles. The first stage, the millisecond timer, counts from 0 to 3 and generates an `msecTimeOut` pulse every 4 clock cycles. This timeout pulse is then used to enable the hundred-millisecond timer, whose counter counts from 0 to 2 and produces a `hundredmsecTimeOut` pulse every 3 pulses of the millisecond timer. Finally, the count-to-10 timer counts from 0 to 1 and generates the final `OneSecTimeout` pulse.

Critical Testing: The waveform clearly shows the hierarchical propagation of timeout signals through all three timer stages. With the shortened simulation terminal counts, the final `OneSecTimeout` pulse is produced every 24 clock cycles, corresponding to the expected nested timing relationship of $4 \times 3 \times 2$. The waveform also shows repeated cycles of counting and timeout generation, confirming that the module operates continuously and correctly while `enable` remains HIGH. These results validate correct reset behavior, proper enable control, accurate intermediate timeout generation, and correct final one-second timeout pulse generation of the OneSecTimer module.

3.5 TwoDigitTimer:



Two-Digit Timer Simulation Results

The simulation results for the Two-Digit Timer module are shown in the waveform screenshot. To verify correct operation, the input signals `clk`, `rst`, `timer_enable`, and `timer_reconfig` are applied while observing the outputs `TensDigit`, `OnesDigit`, and `TimeOut`. The waveform demonstrates that the timer performs a proper two-digit countdown operation, where the ones digit decrements first and the tens digit changes whenever the ones digit completes a full cycle.

The simulation confirms that when the timer is enabled, the countdown progresses in decimal form from higher values toward lower values in the expected sequence. The ones digit decreases step by step, and when it reaches 0, it rolls over and causes the tens digit to

decrement, confirming correct borrow operation between the two digit counters. This shows that the One-Second Timer pulse is correctly driving the two-digit countdown logic.

Critical Testing: Reset behavior is verified by asserting `rst`, which initializes the timer outputs to the starting value. Timer reconfiguration is also tested using `timer_reconfig`, which reloads the timer to its preset value and restarts the countdown sequence correctly. The waveform further shows that the timer only counts when `timer_enable` is asserted, confirming proper enable control. When the timer reaches 00, the `TimeOut` signal is activated, indicating that the countdown has completed successfully. These results validate proper countdown sequencing, correct borrow operation between ones and tens digits, correct reloading behavior, timeout generation, and reliable reset handling of the Two-Digit Timer module.

3.6 Random Number Generator:



Random Number Generator Simulation Results

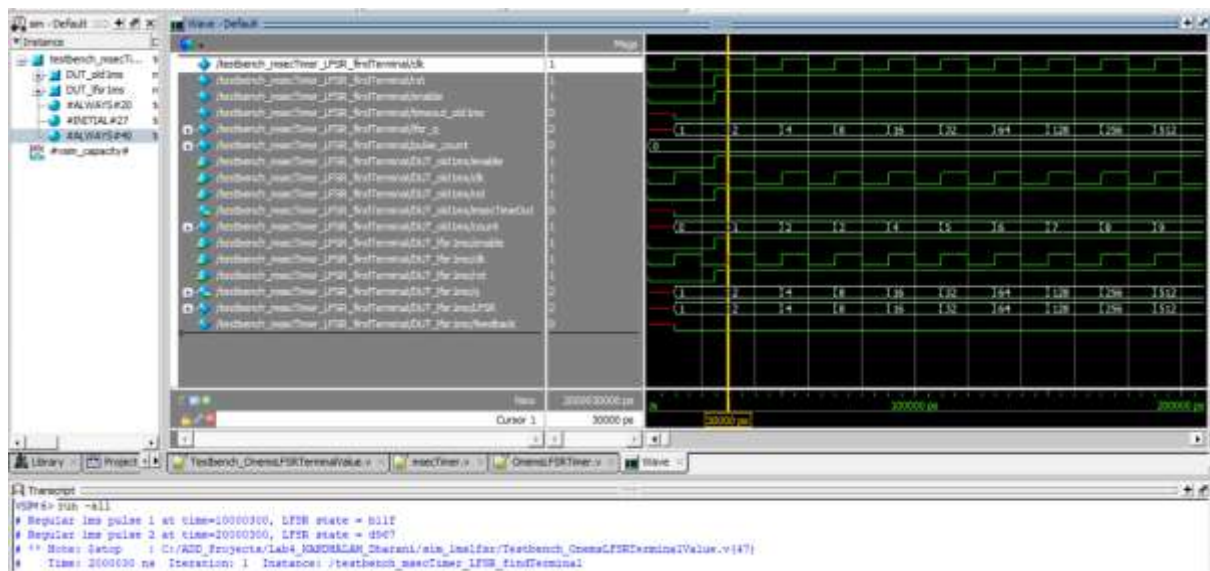
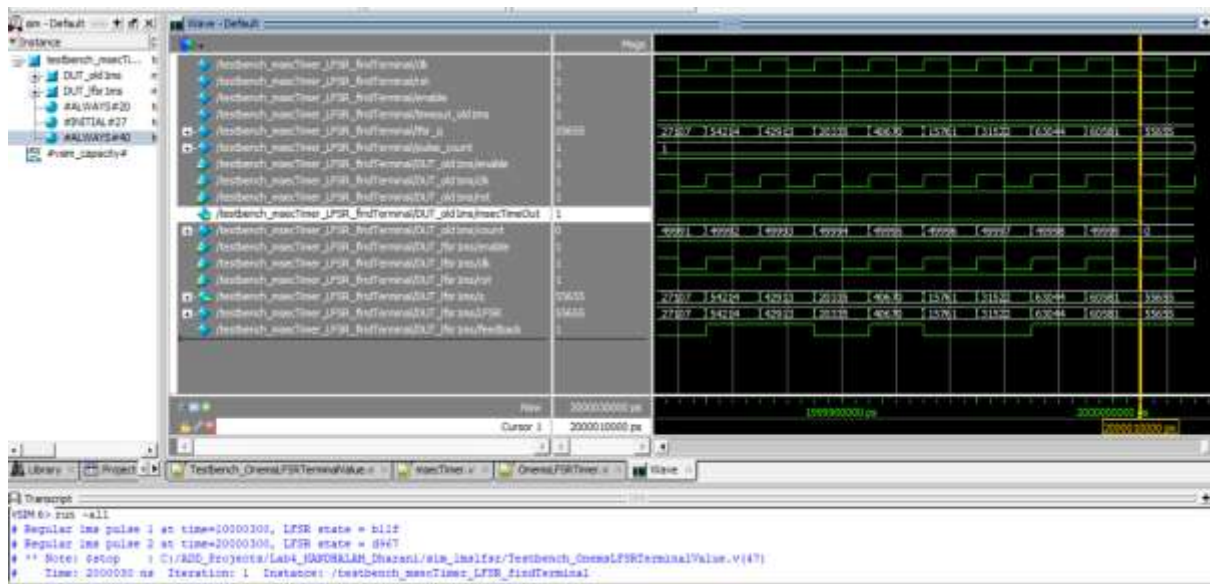
The simulation results for the Random Number Generator module are shown in the waveform screenshot. To verify correct operation, the input signals `clk`, `rst`, and `rngButton` are applied while observing the output `randomNum[3:0]`. The waveform demonstrates that the module generates a 4-bit value based on the duration for which the random number generation button is pressed.

The simulation confirms that at the beginning, when `rst` is asserted, the output `randomNum` is cleared to 0000. After reset is released, pressing the `rngButton` causes the internal counter to start incrementing on each clock cycle. As long as the button remains pressed, the counter continues to count upward in binary sequence. When the button is released, the current counter value is held and becomes the generated random number output.

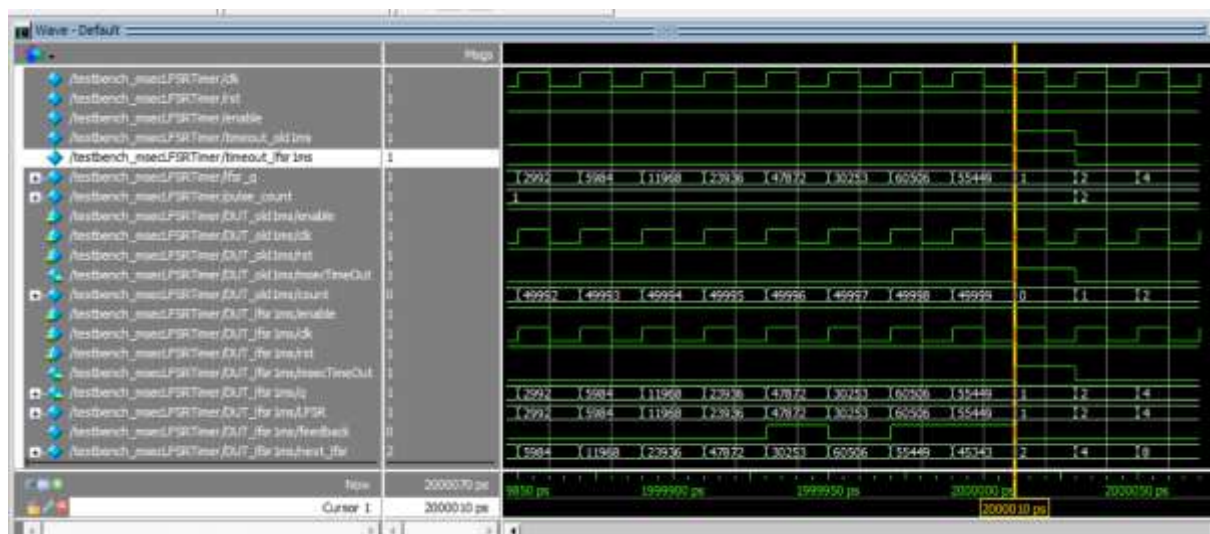
Critical Testing: The waveform shows that while `rngButton` remains active, the output changes through successive values such as 0001, 0010, 0011, 0100, and so on, confirming correct counter operation. When the button is released, the counting stops and the last value is retained at the output, verifying correct capture behavior. Reset functionality is also validated, as asserting `rst` forces the output back to 0000. These results confirm proper reset handling, correct counter-based random number generation, and correct holding of the generated value after button release in the Random Number Generator module.

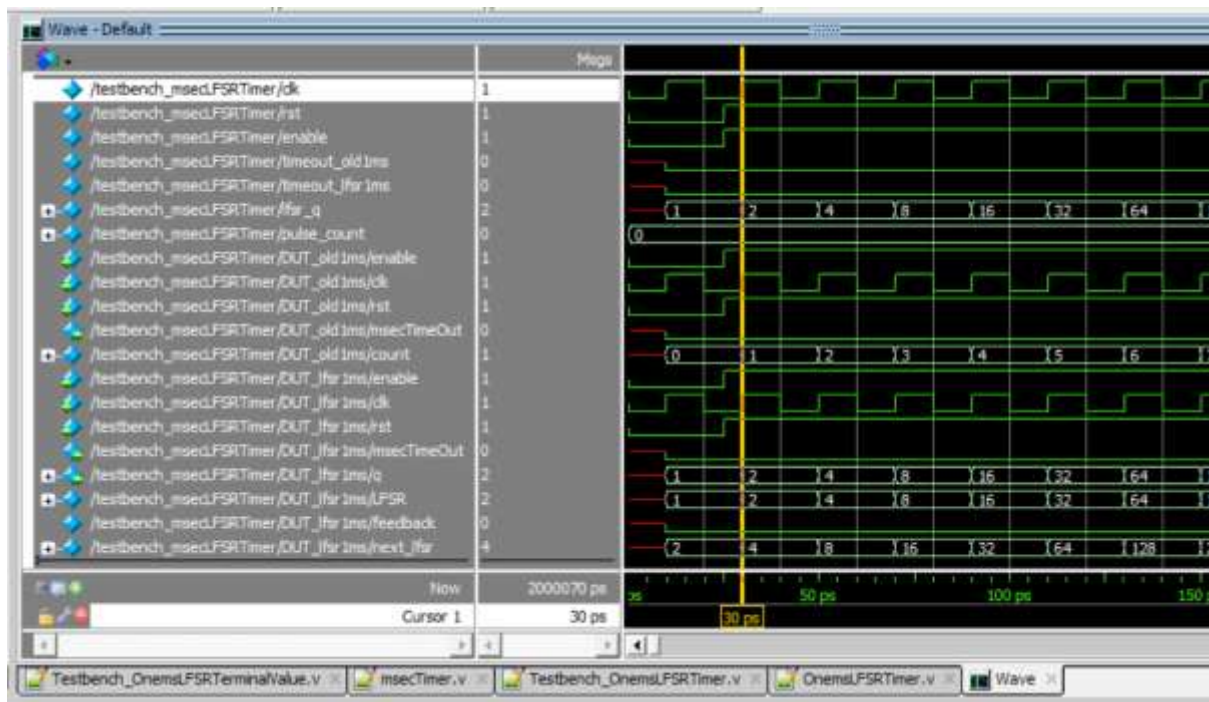
3.7 1ms LFSR Timer:

For the 1st round simulation for terminal Value



The 2nd round simulation side by side with counter 1ms timer:





Simulation Results Description:

The simulation results for the LFSR-based 1 ms timer are shown in the waveform screenshots. To verify correct operation, the regular Lab 3 1 ms timer and the LFSR-based 1 ms timer were simulated side by side using the same clk, rst, and enable signals. In the first simulation run, the regular timer was used as the reference to identify the LFSR terminal state corresponding to 50,000 enabled clock cycles. After the terminal value was determined, the LFSR timer was updated and simulated again for comparison.

The waveform confirms that after reset is released and enable is asserted, both timers begin operating from the same clock reference. The regular timer counts in normal binary order, while the LFSR-based timer advances through pseudo-random LFSR states. Even though the internal state progression is different, both modules generate their timeout pulse at the same instant. The screenshots show that the LFSR state advances from its seed value through successive states, and when the next-state logic reaches the identified terminal condition, the module generates a one-clock-cycle msecTimeOut pulse and restarts the LFSR sequence from the seed.

Critical Testing: The comparison waveform shows that timeout_old1ms and timeout_lfsr1ms go HIGH at the same clock cycle, confirming that the LFSR-based timer matches the timing behavior of the regular 1 ms timer. The waveform also verifies that the timeout pulse is only one clock cycle wide and that the LFSR restarts correctly after timeout. The transcript output reports that both timers matched for two full cycles, confirming correct alignment of the timeout pulses over repeated operation. These results validate proper reset behavior, correct enable control, successful terminal-state detection, correct one-cycle timeout pulse generation, and equivalent timing behavior between the regular and LFSR-based 1 ms timer modules.

4 FPGA Board Testing Results:

The hardware implementation of the system was verified on the DE0-CV FPGA board to confirm correct functionality of authentication, verification, and status indication using LEDs and seven-segment displays.

Case 1: System Initialization After Code Download

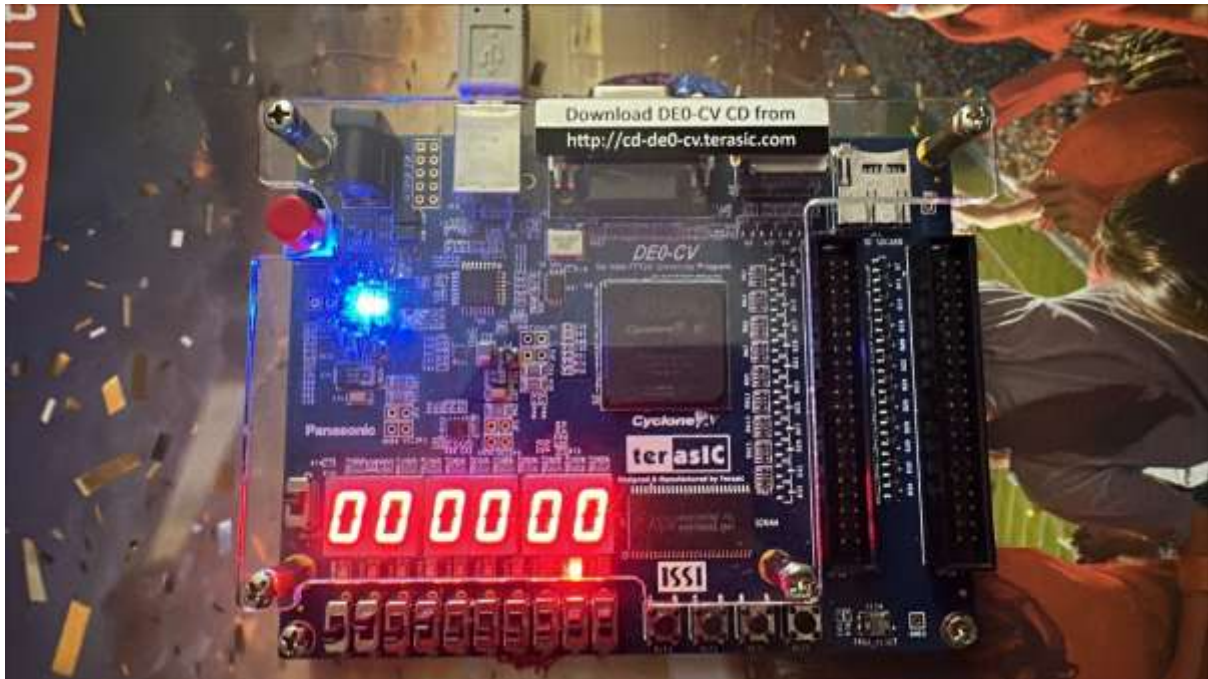


Figure4.1: System Initialization Immediately After Code Download

Just after dumping the code onto the DE0-CV board, the system initializes to its default state. As shown in the image, all seven-segment displays show 0, indicating that no game activity has started yet and no values have been loaded into the system. This confirms that the design powers up correctly in its initial condition before authentication and gameplay begin.

Case 2: Successful Authentication (Logged-In State)

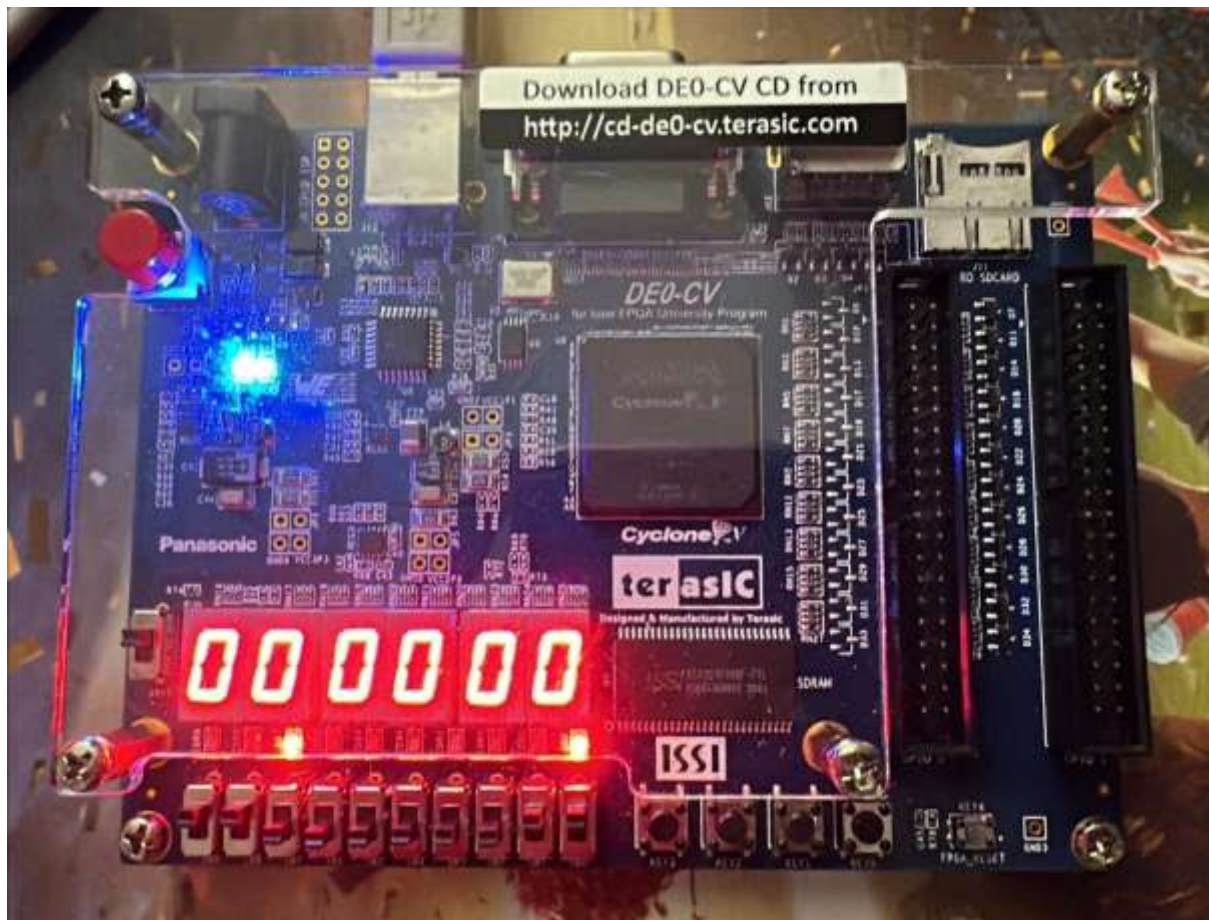


Figure 4.2: System State After Successful Authentication

After entering the correct password, the system transitions from the logged-out state to the logged-in state. As shown in the image, the timer display is initialized to 99, indicating that the system has accepted authentication and loaded the default game time. The logged-in indicator becomes active, confirming that access has been granted and the system is now ready for game start, random number generation, and player input operations.

Case 3: Game Start State



Figure 4.3: Board Showing Game Start After Pressing the Game Button

After pressing the **game start button**, the Level 1 timer starts counting down from **99 seconds**. The changing timer display confirms that the game has entered active mode and the countdown logic is working.

At this stage, the controller generates a hidden target prime, clears the previous guess, and waits for the player's two-digit input.

Case4: Guess Feedback State when guess is larger than Target Pirme



Figure 4.4: Board Showing Guess Feedback State when Guess is larger than Target State

After the game starts, the player enters a two-digit guess using the player switches and load button. In this case, the entered guess is **07**, and the hint display shows **L**, which indicates that the guess is larger than the hidden target prime. The timer continues counting down, and the system remains in active gameplay mode, waiting for the next guess.

Case 5: User Guessed Number is Smaller Than Hidden Target Number

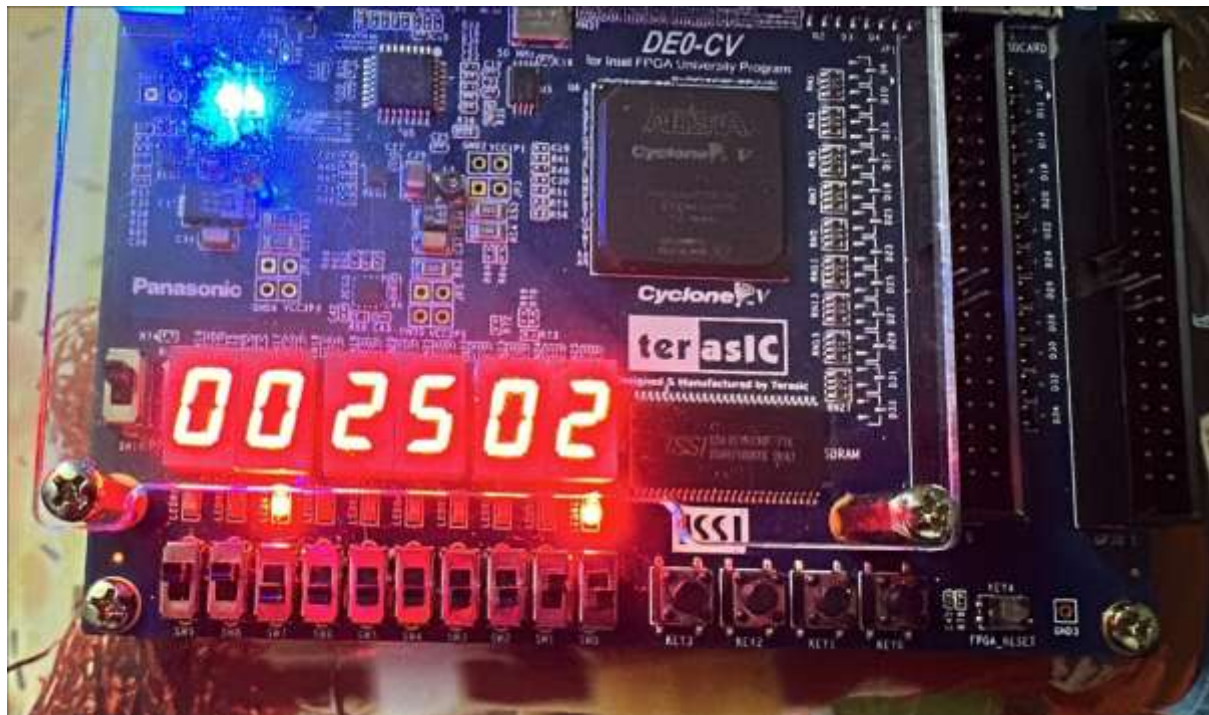


Figure 4.5: Board Showing Guessed Number smaller than Target Number

During active gameplay, the player enters a two-digit guess using the player switches and load button. In this case, the entered guess is **02**, and the hint display shows **S**, which indicates that the guessed prime number is smaller than the hidden target prime. The timer continues counting down, and the system remains in active mode, waiting for the player to enter the next guess.

Case 6: User Entered Number is Not a Prime Number

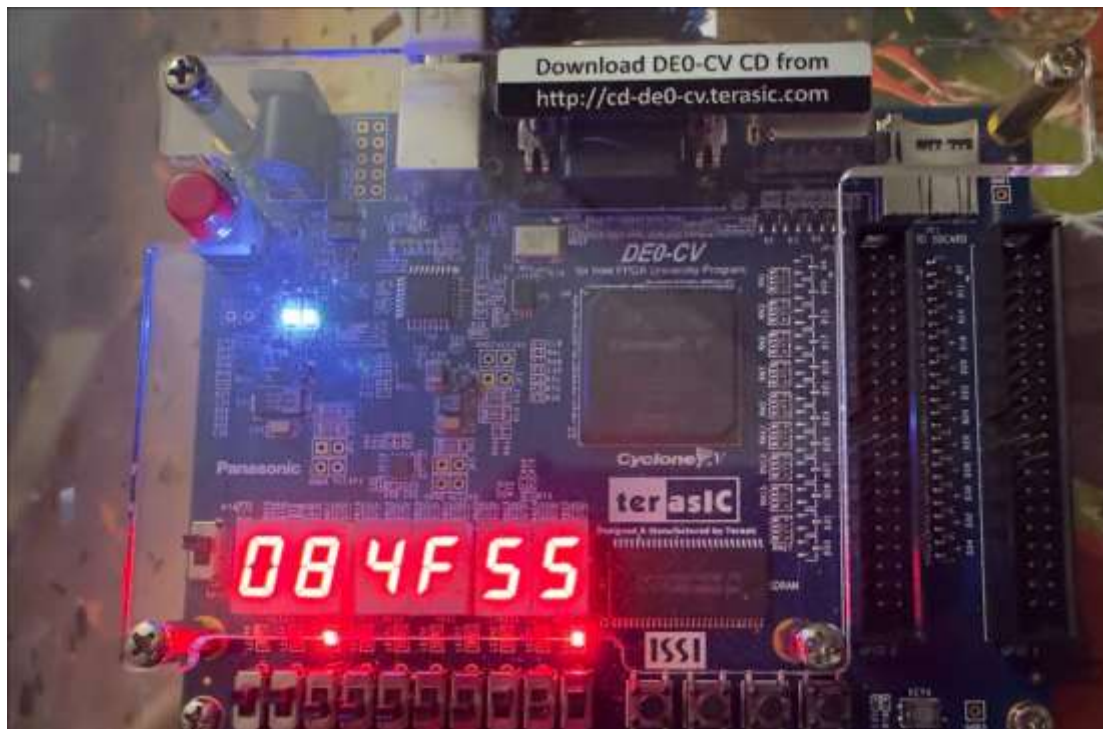


Figure 4.6: Board Showing User Entered Number is Not a Prime Number

During active gameplay, the player enters a two-digit guess using the player switches and load button. In this case, the entered guess is **55**, and the hint display shows **F**. The **F** indication means the entered number is not a valid prime number, so the guess is rejected and the score does not increase. The timer continues counting down, and the system waits for the player to enter another valid prime number guess.

Case 7: User Guessed the Correct Hidden Target Number

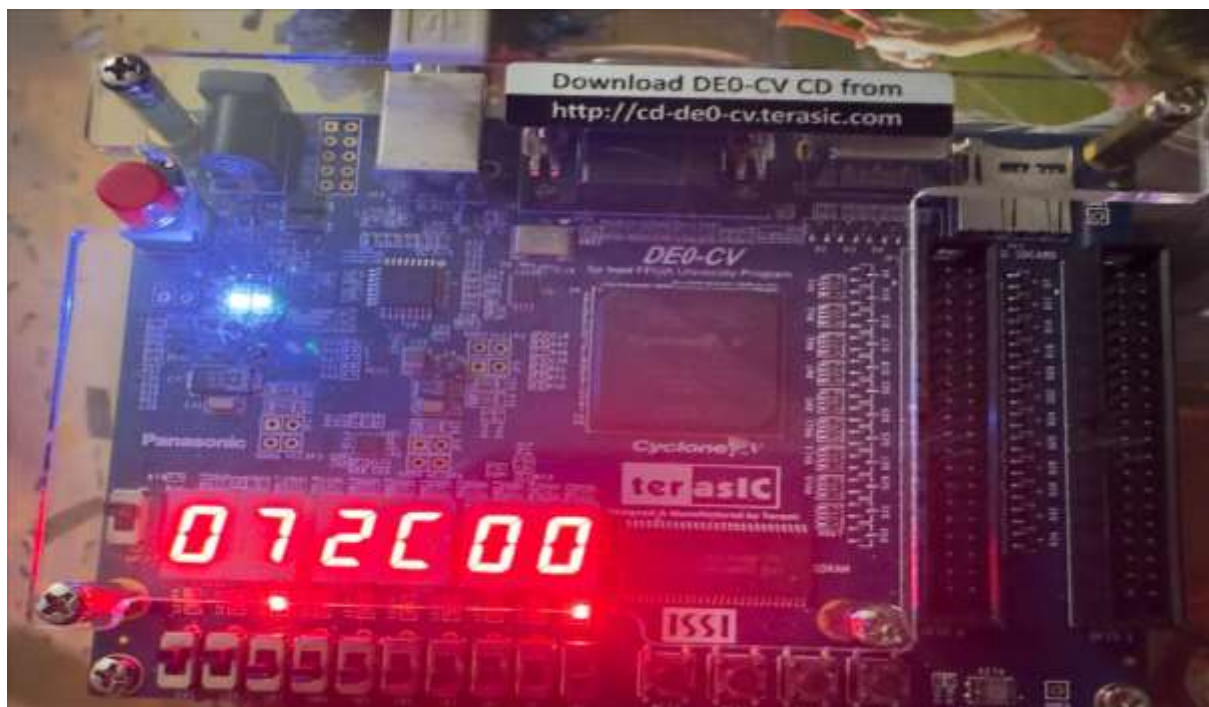


Figure 4.7: Board Showing User Guessed the Correct Hidden Target Number

During active gameplay, the player entered the guess **02** using the player switches and load button. Since **02 matched the hidden target prime**, the hint display shows **C** for correct. After the correct guess is detected, the system increments the score, immediately clears the entered guess back to **00**, and generates a new hidden target prime while the timer continues counting down.

Case 8: Game Over and Final Score Display with winner celebration

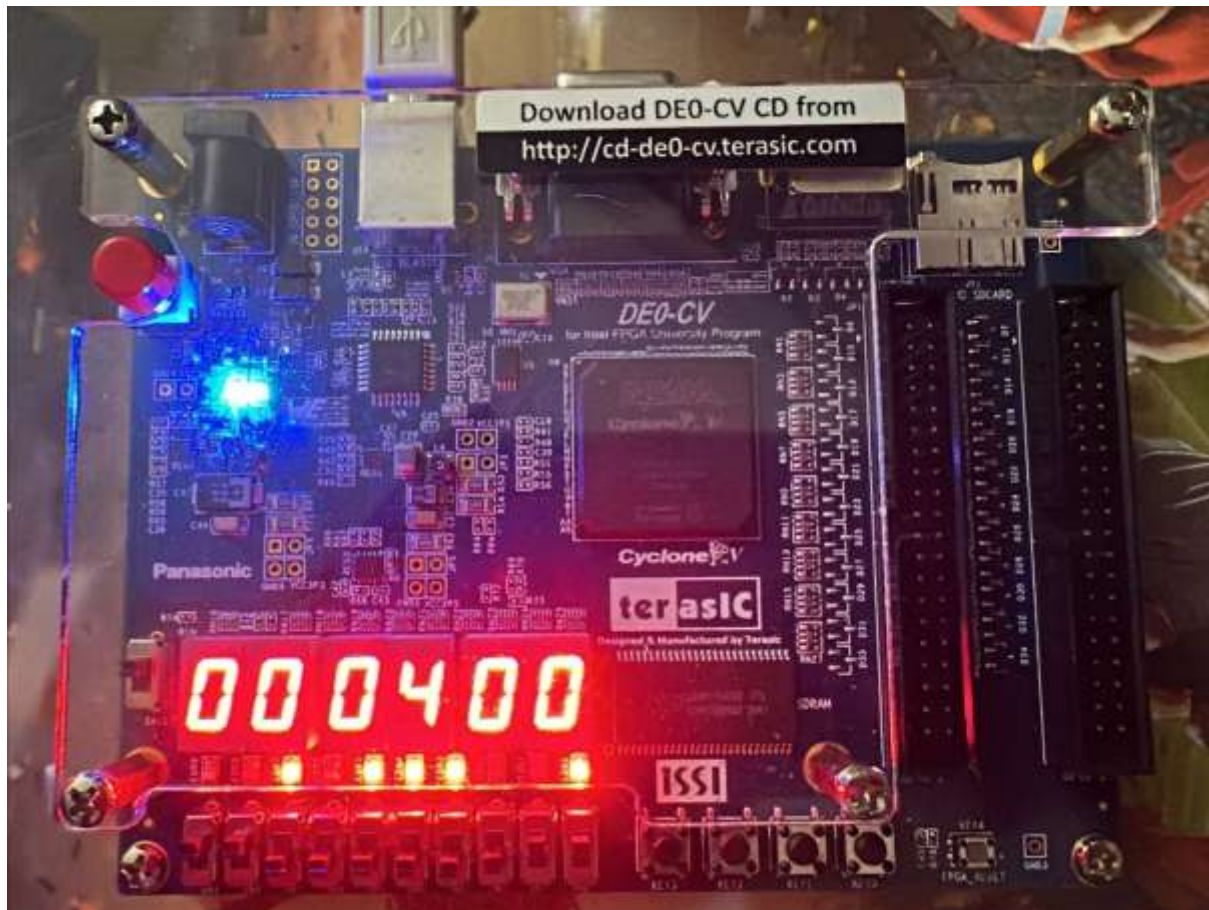


Figure 4.8: Board Showing Game Over and Final Score Display with winner as global winner

When the timer reaches **000**, the game stops and the final score is displayed. In this case, the score is **04**, meaning the player correctly guessed **one hidden prime number** before time expired. Since this is the first round and there are no previous scores stored, the score tracking module treats this score as both the first **local/personal best** and the first **global best**. Therefore, both winner indications are active: **LED4** shows the local winner status, and **LED3, LED4, and LED5** show the global winner status.

Case 9: New User Local Winner but Not Global Winner

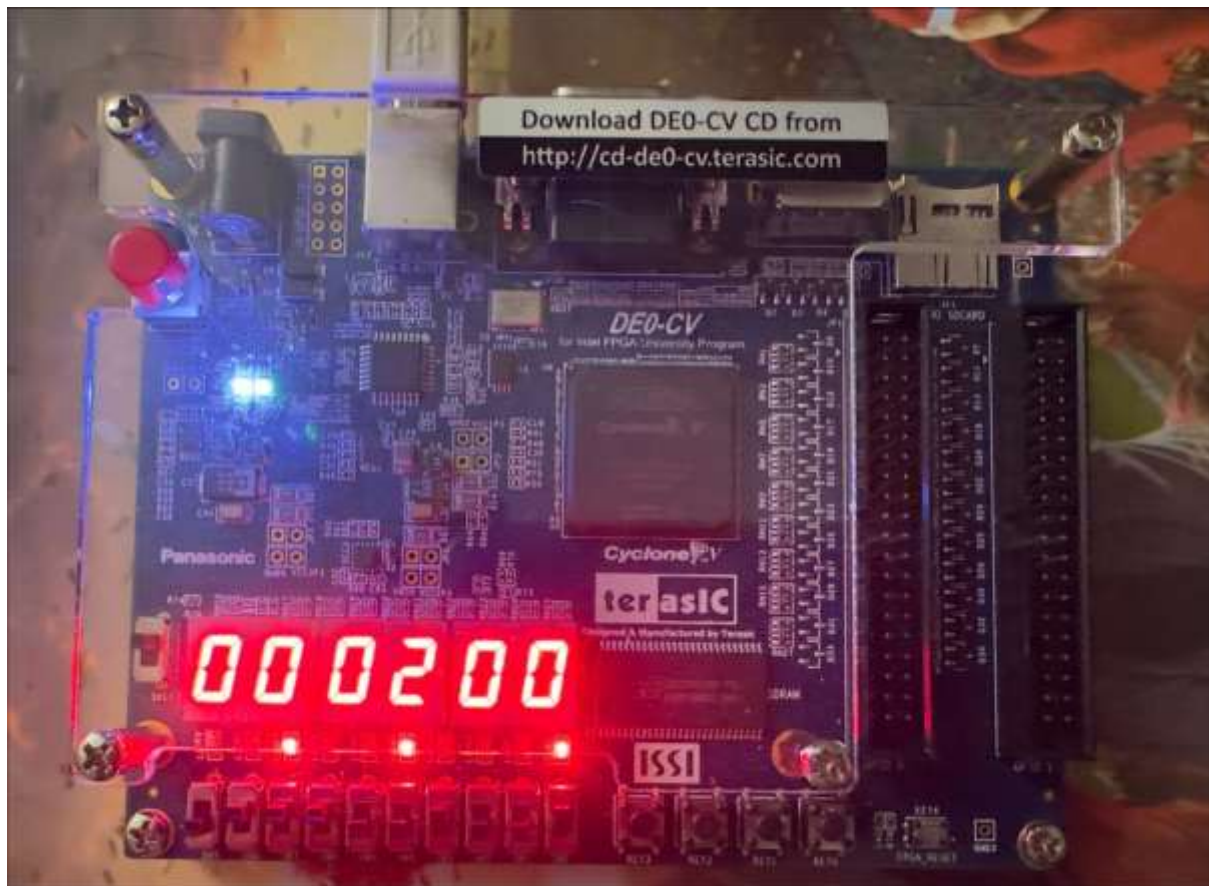


Figure 4.9: Board Showing local winner

game with a final score of **02**. Since this is the first recorded score for this user, the score tracking module marks it as a **local/personal winner** for that player. However, the **global winner** indication is not activated because the previous global benchmark score was **04**, and the current score of **02** is lower than that global best. Therefore, only the **local winner/Personal LED** is shown, while the global winner LEDs remain off.

Case 10: Logout State



Figure 4.10: Board Showing logout state

In this case, the player has logged out after the game is completed. The display is cleared back to the default state, showing **000000**, and the system is no longer in active gameplay mode. The logout status LED is turned on, indicating that the authentication session has ended and the system is ready for another player to log in and start a new game.

5 Video Demo:

5.1 Player1 Demo Links: [Player1_Full_Demo.mp4](#)

5.2 Player2 Demo Link: [Player2_Full_Demo.mp4](#)

5.3 Global and local winners:
[Player2_Global_Local_winners.mp4](#)

5.4 Difficulty levels: [Different_Levels.mp4](#)

6 Conclusion:

In this project, we successfully designed and implemented a complete **FPGA-based Prime Number Guessing Game** using **Verilog HDL**. The system combines several important digital design concepts, including **ROM-based multi-user authentication**,

LFSR-based random prime generation, FSM-based game control, countdown timers, score tracking, LED indicators, and seven-segment display control.

The game starts with a **multi-user login system**, where the player enters a valid player ID and password before accessing the game. After successful authentication, the player selects a difficulty level. Each level changes the prime number range and timer duration, allowing the game to support different levels of challenge.

During gameplay, the FPGA system generates a hidden prime number and shows **G** on the display to indicate that the player can **start guessing**. The player then enters a two-digit guess using switches and the load button. The system gives hints on the seven-segment display to guide the player. If the display shows **S**, the guessed number is **too small** compared to the hidden target number. If it shows **L**, the guessed number is **too large**. If it shows **C**, the guess is **correct**, and the score increases. If it shows **F**, the guess is **wrong or invalid**, such as entering a non-prime number.

The score tracking system records the player's performance and displays the final score after the timer expires. It also identifies **local and global winners** using LED indicators, which makes the game more interactive and competitive.

Overall, this project demonstrates how different Verilog modules can be designed, tested, and integrated into one complete FPGA-based system. It strengthened our understanding of **FSM design, ROM and RAM usage, randomization using LFSR, timing synchronization, display control, and real-time hardware interaction**. The final system is stable, modular, scalable, and can be further improved in the future by adding more users, more difficulty levels, persistent score memory, or additional game modes.

7 Appendix:

Top Module:

```
ROMAuthentication.v  ROM_PSWD.v  GameController_Lab4.v  OneSecTimer.v  Lab4_KANDHALAM_Dharani.qsf  Lab4_KANDHALAM_Dharani.v
1  // ECE6370
2  // Author: Dharani Kandharam - 1389
3  // Name of Module: Lab4Top_KANDHALAM_Dharani
4  // Description: This top module integrates the Lab 4 single-player game.
5  // It separates ROM-based password authentication from game control,
6  // replaces the regular 1 ms timer with the LFSR-based timer, and keeps
7  // the Lab 3 user interface behavior the same.
8  // Comments/Log: Required Lab 4 top-level integration.
9
10 module Lab4_KANDHALAM_Dharani(
11     Player_Switch,
12     Password_Switch,
13     PasswordGameStartButton,
14     Player_LoadButton,
15     RNG_Gen_Button,
16     clk,
17     rst,
18     TimerTens_Display,
19     TimerOnes_Display,
20     ScoreTens_Display,
21     SumOrScoreOnes_Display,
22     Random_Display,
23     Player_Display,
24     Logged_In_LED,
25     Logged_Out_LED,
26     LEDs
27 );
28
29     input  [3:0] Player_Switch;
30     input  [3:0] Password_Switch;
31     input          PasswordGameStartButton;
32     input          Player_LoadButton;
33     input          RNG_Gen_Button;
34     input          clk, rst;
35
36     output [6:0] TimerTens_Display, TimerOnes_Display;
37     output [6:0] ScoreTens_Display, SumOrScoreOnes_Display;
38     output [6:0] Random_Display, Player_Display;
39     output          Logged_In_LED, Logged_Out_LED;
40     output [9:0] LEDs;
41
```

```
ROMAuthentication.v  ROM_PSWD.v  GameController_Lab4.v  OneSecTimer.v
40      output [9:0] LEDs;
41
42      wire GameButtonPulse;
43      wire Load_P_In;
44
45      wire B_password;
46      wire B_game;
47
48      assign B_password = GameButtonPulse;
49      assign B_game     = GameButtonPulse;
50
51      wire Passed;
52      wire Logged_In, Logged_Out;
53      wire Load_P_Out;
54      wire RNG_Gen_Out;
55      wire Timer_Enable, Timer_Reconfig;
56      wire ShowScore;
57      wire LogoutPulse;
58
59      wire [3:0] Player_Number;
60      wire [3:0] Random_Number;
61      wire [3:0] Sum;
62
63      wire oneSecPulse;
64      wire [3:0] Tensdigit, Onesdigit;
65      wire TimeOut;
66
67      wire MatchLED, NonMatchLED;
68      wire [3:0] ScoreTens, ScoreOnes;
69
70      wire [3:0] MiddleLeftValue;
71      wire [3:0] MiddleRightValue;
72      wire [3:0] RandomDisplayValue;
73      wire [3:0] PlayerDisplayValue;
74
75      ButtonShaper PasswordGameButtonShaper(
76          PasswordGameStartButton,
77          GameButtonPulse,
78          clk,
79          rst
80      );
81
```

Verilog file

```

ROMAuthentication.v  ROM_PSWD.v  GameController_Lab4.v  OneSecTimer.v
82  ButtonShaper PlayerLoadButtonShaper(
83      Player_LoadButton,
84      Load_P_In,
85      clk,
86      rst
87  );
88
89  ROMAuthentication AuthenticationModule(
90      .clk(clk),
91      .rst(rst),
92      .B_password(B_password),
93      .LogoutPulse(LogoutPulse),
94      .PasswordSwitches(Password_Switch),
95      .Passed(Passed),
96      .Logged_In(Logged_In),
97      .Logged_Out(Logged_Out)
98  );
99
100  GameController_Lab4 GameControllerModule(
101      .clk(clk),
102      .rst(rst),
103      .B_game(B_game),
104      .B_load(Load_P_In),
105      .Passed(Passed),
106      .Logged_In(Logged_In),
107      .Timeout(Timeout),
108      .RNG_Gen_In(RNG_Gen_Button),
109      .Load_P_In(Load_P_In),
110      .Timer_Reconfig(Timer_Reconfig),
111      .Timer_Enable(Timer_Enable),
112      .RNG_Gen_Out(RNG_Gen_Out),
113      .Load_P_Out(Load_P_Out),
114      .ShowScore(ShowScore),
115      .LogoutPulse(LogoutPulse)
116  );
117
118  LoadRegister PlayerLoadRegister(
119      Player_Switch,
120      Player_Number,
121      clk,
122      rst,
123      Load_P_Out

```

Verilog file

ROMAuthentication.v	ROM_PSWD.v	GameController_Lab4.v	OneSecTimer.v
---------------------	------------	-----------------------	---------------

```

21         clk,
22         rst,
23         Load_P_Out
24     );
25
26     RandomNumberGenerator RNGModule(
27         Random_Number,
28         RNG_Gen_Out,
29         clk,
30         rst
31     );
32
33     OneSecTimer LFSR_OneSecondTimerModule(
34         Timer_Enable,
35         clk,
36         rst,
37         oneSecPulse
38     );
39
40     TwoDigitTimer TimerModule(
41         Tensdigit,
42         Onesdigit,
43         TimeOut,
44         oneSecPulse,
45         Timer_Reconfig,
46         clk,
47         rst
48     );
49
50     My4BAdder AdderModule(
51         Player_Number,
52         Random_Number,
53         Sum
54     );
55
56     VerificationModule VerificationModule1(
57         Sum,
58         MatchLED,
59         NonMatchLED
60     );
61
62     Scoreboard ScoreboardModule(

```

log file

```

162 Scoreboard ScoreboardModule (
163     ScoreTens,
164     ScoreOnes,
165     Load_P_Out,
166     MatchLED,
167     TimeOut,
168     Timer_Reconfig,
169     clk,
170     rst
171 );
172
173 assign RandomDisplayValue = (Logged_In == 1'b1) ? Random_Number : 4'd0;
174 assign PlayerDisplayValue = (Logged_In == 1'b1) ? Player_Number : 4'd0;
175
176 assign MiddleLeftValue = (ShowScore == 1'b1) ? ScoreTens : 4'd0;
177 assign MiddleRightValue = (ShowScore == 1'b1) ? ScoreOnes :
178     ((Logged_In == 1'b1) ? Sum : 4'd0);
179
180 My7SegmentDecoder_4To7 TimerTensDisplayModule (
181     Tensdigit,
182     TimerTens_Display
183 );
184
185 My7SegmentDecoder_4To7 TimerOnesDisplayModule (
186     Onesdigit,
187     TimerOnes_Display
188 );
189
190 My7SegmentDecoder_4To7 ScoreTensDisplayModule (
191     MiddleLeftValue,
192     ScoreTens_Display
193 );
194
195 My7SegmentDecoder_4To7 SumOrScoreOnesDisplayModule (
196     MiddleRightValue,
197     SumOrScoreOnes_Display
198 );
199
200 My7SegmentDecoder_4To7 RandomDisplayModule (
201     RandomDisplayValue,
202     Random_Display
203 );

```

ROMAuthentication.v	ROM_PSWD.v	GameController_lab4.v	OneSecTimer.v	Lab4_KANDH
---------------------	------------	-----------------------	---------------	------------

```

192         ScoreTens_Display
193     );
194
195     My7SegmentDecoder_4To7 SumOrScoreOnesDisplayModule (
196         MiddleRightValue,
197         SumOrScoreOnes_Display
198     );
199
200     My7SegmentDecoder_4To7 RandomDisplayModule (
201         RandomDisplayValue,
202         Random_Display
203     );
204
205     My7SegmentDecoder_4To7 PlayerDisplayModule (
206         PlayerDisplayValue,
207         Player_Display
208     );
209
210     assign Logged_In_LED = Logged_In;
211     assign Logged_Out_LED = Logged_Out;
212
213     assign LEDs[9] = NonMatchLED;
214     assign LEDs[8] = MatchLED;
215     assign LEDs[0] = Logged_In;
216     assign LEDs[1] = Logged_Out;
217     assign LEDs[7:2] = 6'b0;
218
219 endmodule

```

LoadRegister Module:

```
1 // ECE6370 : ADO
2 // Author: Dharani Kandhalaan - 1369
3 //
4 // LoadRegister
5 // This module stores a 4-bit value when the load signal is high.
6 // The register updates its output only when the LOAD button is
7 // pressed, preventing intermediate switch changes from affecting
8 // the system.
9 //
10 // • If load = 1 → input value is stored in register.
11 // • If load = 0 → previous value is retained.
12 // • Active-low reset clears the register to 0.
13 //
14 // Used in Lab2 to store Player 1 and Player 2 numbers only after
15 // the corresponding LOAD button is pressed.
16 //
17 module LoadRegister(Reg_Input, Reg_Output, clk, Rst, load)
18   input [3:0] Reg_Input;
19   output [3:0] Reg_Output;
20   input clk, Rst, load;
21
22   reg [3:0] Reg_Output;
23
24   always @(posedge clk)
25     begin
26       if(Rst == 1'b0)
27         begin
28           //reset condition
29           Reg_Output <= 4'b0000;
30         end
31       else
32         //normal operation after reset where it follows input
33         begin
34           if(load == 1'b1) //whenever load button is pushed we are sending input to output
35             begin
36               Reg_Output <= Reg_Input;
37             end
38         end
39     end
40 endmodule
```



```

testbench_LoadRegister.v new 6 new 7 new 8 AccessController.v
28      #5 Rst = 1'b0;
29      @(posedge clk);
30      @(posedge clk);
31      @(posedge clk);
32      @(posedge clk);
33      //pulled the reset button backup
34      #5 Rst = 1'b1;
35      @(posedge clk);
36      @(posedge clk);
37      @(posedge clk);
38      @(posedge clk);
39      #5 Reg_Input = 4'b0001;
40      @(posedge clk);
41      #5 load = 1'b1;
42      @(posedge clk);
43      //pushing back the load button to zero for next operations
44      #5 load = 1'b0;
45      @(posedge clk);
46      #5 Reg_Input = 4'b1111;
47      @(posedge clk);
48      @(posedge clk);
49      @(posedge clk);
50      #5 Reg_Input = 4'b0111;
51      @(posedge clk);
52      @(posedge clk);
53      #5 load = 1'b1;
54      @(posedge clk);
55      #5 load = 1'b0;
56      @(posedge clk);
57
58      end
59
60      endmodule

```

ButtonShaper:

```

1  ECE6370 : ADD
2  // Author: Bharani Kandham - 1309
3  // ButtonShaper
4  //
5  // ButtonShaper
6  //
7  // This module converts a mechanical push button input into a
8  // single clock-cycle pulse.
9  //
10 //
11 // • Push buttons may produce multiple transitions due to bouncing.
12 // • This FSM ensures only one clean pulse is generated per press.
13 // • Output (B_out) goes high for exactly one clock cycle when the
14 //   button is pressed.
15 //
16 // States:
17 // INIT -> waits for button press
18 // PULSE -> generates one clock pulse
19 // WAIT -> waits until button is released
20 //
21 module ButtonShaper(B_in, B_out, clk, rst);
22
23     input B_in;
24     output B_out;
25     input clk, rst;
26     reg B_out;
27
28     parameter INIT = 0, PULSE = 1, WAIT = 2;
29
30     reg [2:0] State, StateNext;
31
32     // Combinational Logic using blocking assignment
33     always @(B_in) begin
34         case (State)
35             INIT: begin
36                 B_out = 1'b0;
37                 if (B_in == 1'b0)
38                     StateNext = PULSE;
39                 else
40                     StateNext = INIT;

```

```

34
35 INIT: begin
36     B_out = 1'b0;
37     if (B_in == 1'b0)
38         StateNext = PULSE;
39     else
40         StateNext = INIT;
41     end
42
43 PULSE: begin
44     B_out = 1'b1;
45     StateNext = WAIT;
46     end
47
48 WAIT: begin
49     B_out = 1'b0;
50     if (B_in == 1'b1)
51         StateNext = INIT;
52     else
53         StateNext = WAIT;
54     end
55
56 default: begin
57     B_out = 1'b0;
58     StateNext = INIT;
59     end
60
61 endcase
62 end
63
64 // StateRegister using non-blocking assignment
65 always @(posedge clk) begin
66     if (rst == 1'b0)
67         State <= INIT;
68     else
69         State <= StateNext;
70     end
71
72 endmodule
73

```

```
testbench_loadRegister.v testbench_ButtonShaper.v AccessController.v AccessController.v
1 `timescale 1 ns/100 ps
2 module testbench_ButtonShaper();
3
4     reg clk, rst, B_in;
5     wire B_out;
6
7     //clock
8     always
9     begin
10         clk = 1'b0;
11         #10;
12         clk = 1'b1;
13         #10;
14     end
15
16     ButtonShaper DUT_MyButtonShaper(B_in, B_out, clk, rst);
17
18     initial
19     begin
20         rst = 1'b1; // inactive (since reset is active-low)
21         B_in = 1'b1; // start high -> INIT will stay in INIT
22
23         @(posedge clk); // forces the signal to next rising edge of the clock
24         @(posedge clk);
25         @(posedge clk);
26         //Reset
27         //it pulls down the signal 5ns after the clock edge
28         #5 rst = 1'b0; //press reset
29         @(posedge clk);
30         @(posedge clk);
31         @(posedge clk);
32
33         //pulled the reset button backup
34         #5 rst = 1'b1;
35         @(posedge clk);
36         @(posedge clk);
37
38         // ---- Test 1: INIT -> PULSE -> WAIT (B_in goes 0) ----
39         // INIT stays while B_in=1, then B_in=0 triggers PULSE on next clock
40         #5 B_in = 1'b0;
41         // hold for 10 cycles
42         @(posedge clk);
43         @(posedge clk);
44         @(posedge clk);
45         @(posedge clk);
46     end
47 endmodule
```

Verilog file length: 2.211 lines: 84

```
testbench_LoadRegister.v testbench_ButtonShaper.v AccessController.v
43         @ (posedge clk) ;
44     @ (posedge clk) ;
45         @ (posedge clk) ;
46         @ (posedge clk) ;
47         @ (posedge clk) ;
48         @ (posedge clk) ;
49         @ (posedge clk) ;
50         @ (posedge clk) ;
51         @ (posedge clk) ;
52
53
54         // release button back to 1
55     #5 B_in = 1'b1;
56         @ (posedge clk) ;
57         @ (posedge clk) ;
58
59         // Another long press example (20 cycles)
60     #5 B_in = 1'b0;
61         @ (posedge clk) ;
62         @ (posedge clk) ;
63         @ (posedge clk) ;
64         @ (posedge clk) ;
65         @ (posedge clk) ;
66     @ (posedge clk) ;
67         @ (posedge clk) ;
68         @ (posedge clk) ;
69         @ (posedge clk) ;
70         @ (posedge clk) ;
71         @ (posedge clk) ;
72         @ (posedge clk) ;
73         @ (posedge clk) ;
74
75         //release the button
76     #5 B_in = 1'b1;
77         @ (posedge clk) ;
78         @ (posedge clk) ;
79
80
81     end
82
83 endmodule
84
```

Verilog file length

ROM Authentication:

```

1  // ECE6370
2  // Author: Dharani Kandharam - 1389
3  // Name of Module: ROMAuthentication
4  // Description: This module authenticates the user by comparing the entered
5  // password digits against the default password stored in ROM. The user enters
6  // one digit at a time using PasswordSwitches[3:0] and presses B_password for
7  // each digit. The FSM fetches the expected digit from ROM, waits for ROM data,
8  // captures it, compares it with the entered digit, and repeats for all four
9  // digits. If all digits match, Passed is asserted and the system enters the
10 // logged-in state. Bonus feature: LogoutPulse returns the system to the
11 // logged-out state.
12 // Comments/Log: Lab 4 ROM-based authentication with logout support.
13
14 module ROMAuthentication(
15     input        clk,
16     input        rst,
17     input        B_password,
18     input        LogoutPulse,           // bonus: logout request
19     input [3:0] PasswordSwitches,
20     output reg    Passed,
21     output reg    Logged_In,
22     output reg    Logged_Out
23 );
24
25     reg [4:0] ROMAddr;
26     wire [3:0] ROMData;
27
28     ROM_PSWD DUT_ROM_PSWD (
29         .address(ROMAddr),
30         .clock(clk),
31         .q(ROMData)
32     );
33
34     reg [3:0] State;
35     reg [3:0] NextState;
36
37     localparam S_CHECK_BUTTON = 4'd0,
38                S_FETCH_ROM    = 4'd1,
39                S_ROM_CYC1     = 4'd2,
40                S_ROM_CYC2     = 4'd3,
41                S_ROM_CATCH    = 4'd4,
42                S_COMPARE      = 4'd5

```

```
ROMAuthentication.v ROM_PSWD.v GameController_Lab4.v OneSecTimer.v Lab4_KANDHALAM_Dharani.qsf
42 S_COMPARE = 4'd5,
43 S_VERIFY = 4'd6,
44 S_SUCCESS = 4'd7,
45 S_FAIL = 4'd8;
46
47 reg [1:0] Cnt;
48 reg [3:0] Player_digit;
49 reg [3:0] ROM_digit;
50 reg MatchOK;
51
52 always @(posedge clk) begin
53     if (rst == 1'b0) begin
54         State <= S_CHECK_BUTTON;
55         ROMAddr <= 5'd0;
56         Cnt <= 2'd0;
57         Player_digit <= 4'd0;
58         ROM_digit <= 4'd0;
59         MatchOK <= 1'b0;
60         Passed <= 1'b0;
61         Logged_In <= 1'b0;
62         Logged_Out <= 1'b1;
63     end
64     else if (LogoutPulse == 1'b1) begin
65         State <= S_CHECK_BUTTON;
66         ROMAddr <= 5'd0;
67         Cnt <= 2'd0;
68         Player_digit <= 4'd0;
69         ROM_digit <= 4'd0;
70         MatchOK <= 1'b0;
71         Passed <= 1'b0;
72         Logged_In <= 1'b0;
73         Logged_Out <= 1'b1;
74     end
75     else begin
76         State <= NextState;
77
78         case (State)
79             S_CHECK_BUTTON: begin
80                 if (Passed == 1'b0 && B_password == 1'b1) begin
81                     Player_digit <= PasswordSwitches;
82                     ROMAddr <= {3'b000, Cnt};
83                 end
84             end
85         endcase
86     end
87 end
```

```
ROMAuthentication.v ROM_PSWD.v GameController_Lab4.v OneSecTimer.v Lab4_KANDH
81 Player_digit <= PasswordSwitches;
82 ROMAddr <= {3'b000, Cnt};
83 end
84 end
85
86 S_FETCH_ROM: begin
87     ROMAddr <= {3'b000, Cnt};
88 end
89
90 S_ROM_CYC1: begin
91     ROMAddr <= {3'b000, Cnt};
92 end
93
94 S_ROM_CYC2: begin
95     ROMAddr <= {3'b000, Cnt};
96 end
97
98 S_ROM_CATCH: begin
99     ROM_digit <= ROMData;
100 end
101
102 S_COMPARE: begin
103     if (Player_digit == ROM_digit)
104         MatchOK <= 1'b1;
105     else
106         MatchOK <= 1'b0;
107 end
108
109 S_VERIFY: begin
110     if (MatchOK == 1'b1 && Cnt != 2'd3)
111         Cnt <= Cnt + 2'd1;
112 end
113
114 S_SUCCESS: begin
115     Passed <= 1'b1;
116     Logged_In <= 1'b1;
117     Logged_Out <= 1'b0;
118     Cnt <= 2'd0;
119 end
120
121 S_FAIL: begin
122     Passed <= 1'b0;
```

```
ROMAuthentication.v ROM_PSWD.v GameController_Lab4.v OneSecTimer.v
120
121         S_FAIL: begin
122             Passed      <= 1'b0;
123             Logged_In   <= 1'b0;
124             Logged_Out  <= 1'b1;
125             Cnt         <= 2'd0;
126             MatchOK     <= 1'b0;
127         end
128
129         default: begin
130             end
131         endcase
132     end
133 end
134
135 always @(*) begin
136     NextState = State;
137
138     case (State)
139     S_CHECK_BUTTON: begin
140         if (Passed == 1'b1)
141             NextState = S_SUCCESS;
142         else if (B_password == 1'b1)
143             NextState = S_FETCH_ROM;
144         else
145             NextState = S_CHECK_BUTTON;
146         end
147
148     S_FETCH_ROM: begin
149         NextState = S_ROM_CYC1;
150     end
151
152     S_ROM_CYC1: begin
153         NextState = S_ROM_CYC2;
154     end
155
156     S_ROM_CYC2: begin
157         NextState = S_ROM_CATCH;
158     end
159
160     S_ROM_CATCH: begin
161         NextState = S_COMPARE;
```

```
ROMAuthentication.v ROM_PSWD.v GameController_Lab4.v OneSecTimer.v Lab4
153         nextState = S_ROM_CYC2;
154     end
155
156     S_ROM_CYC2: begin
157         nextState = S_ROM_CATCH;
158     end
159
160     S_ROM_CATCH: begin
161         nextState = S_COMPARE;
162     end
163
164     S_COMPARE: begin
165         nextState = S_VERIFY;
166     end
167
168     S_VERIFY: begin
169         if (MatchOK == 1'b0)
170             nextState = S_FAIL;
171         else if (Cnt == 2'd3)
172             nextState = S_SUCCESS;
173         else
174             nextState = S_CHECK_BUTTON;
175     end
176
177     S_SUCCESS: begin
178         nextState = S_SUCCESS;
179     end
180
181     S_FAIL: begin
182         nextState = S_CHECK_BUTTON;
183     end
184
185     default: begin
186         nextState = S_CHECK_BUTTON;
187     end
188 endcase
189 end
190
191 endmodule
```

Game Controller:

```

1  // ECE6370
2  // Author: Dharani Kandharam - 1389
3  // Name of Module: GameController_Lab4
4  // Description: This module controls the game flow after authentication is
5  // completed. It is separate from the ROM-based authentication module.
6  // After Passed becomes HIGH, the controller issues a one-cycle timer
7  // reconfiguration pulse, waits for the shared game button press to start
8  // the timer, enables gameplay during the active session, blocks gameplay
9  // when timeout occurs, and waits for the shared game button press to
10 // restart the game. Bonus feature: if the user is logged in and the timer
11 // is not running, pressing the Player Load button logs the user out.
12 // Comments/Log: Lab 4 version with logout bonus.
13
14 module GameController_Lab4(
15     input  clk,
16     input  rst,
17     input  B_game,          // shaped shared button
18     input  B_load,          // shaped player's load button
19     input  Passed,          // from authentication module
20     input  Logged_In,       // from authentication module
21     input  Timeout,         // from two-digit timer
22     input  RNG_Gen_In,      // raw/allowed RNG request from board path
23     input  Load_P_In,      // shaped player load request
24     output reg Timer_Reconfig,
25     output reg Timer_Enable,
26     output reg RNG_Gen_Out,
27     output reg Load_P_Out,
28     output reg ShowScore,
29     output reg LogoutPulse
30 );
31
32     reg [2:0] State;
33     reg [2:0] NextState;
34
35     localparam S_INIT          = 3'd0,
36                S_PASSED        = 3'd1,
37                S_WAIT_FOR_GAMESTART = 3'd2,
38                S_GAMEPLAY      = 3'd3,
39                S_GAMEOVER       = 3'd4;
40
41     always @(posedge clk) begin
42         if (rst == 1'b0)

```

Verilog file

```

ROMAuthentication.v  ROM_PSWD.v  GameController_Lab4.v  OneSecTimer.v  Lab4_KANDHALAM_Dharani
43         State <= S_INIT;
44     else
45         State <= NextState;
46     end
47
48     always @(*) begin
49         NextState = State;
50
51         case (State)
52             S_INIT: begin
53                 if (Passed == 1'b1)
54                     NextState = S_PASSED;
55                 else
56                     NextState = S_INIT;
57             end
58
59             S_PASSED: begin
60                 NextState = S_WAIT_FOR_GAMESTART;
61             end
62
63             S_WAIT_FOR_GAMESTART: begin
64                 if (Passed == 1'b0)
65                     NextState = S_INIT;
66                 else if (Logged_In == 1'b1 && B_load == 1'b1)
67                     NextState = S_INIT; // logout
68                 else if (B_game == 1'b1)
69                     NextState = S_GAMEPLAY;
70                 else
71                     NextState = S_WAIT_FOR_GAMESTART;
72             end
73
74             S_GAMEPLAY: begin
75                 if (Passed == 1'b0)
76                     NextState = S_INIT;
77                 else if (TimeOut == 1'b1)
78                     NextState = S_GAMEOVER;
79                 else
80                     NextState = S_GAMEPLAY;
81             end
82
83             S_GAMEOVER: begin
84                 if (Passed == 1'b0)

```

```

ROMAuthentication.v ROM_PSWD.v GameController_Lab4.v OneSecTimer.v Lab4_KANDHALAM_Dha
85         nextState = S_INIT;
86     else if (Logged_In == 1'b1 && B_load == 1'b1)
87         nextState = S_INIT; // logout
88     else if (B_game == 1'b1)
89         nextState = S_PASSED; // restart flow
90     else
91         nextState = S_GAMEOVER;
92     end
93
94     default: begin
95         nextState = S_INIT;
96     end
97 endcase
98 end
99
100 always @(*) begin
101     Timer_Reconfig = 1'b0;
102     Timer_Enable   = 1'b0;
103     RNG_Gen_Out    = 1'b0;
104     Load_P_Out     = 1'b0;
105     ShowScore      = 1'b0;
106     LogoutPulse    = 1'b0;
107
108     case (State)
109     S_INIT: begin
110         // everything blocked
111     end
112
113     S_PASSED: begin
114         Timer_Reconfig = 1'b1; // one-cycle pulse
115     end
116
117     S_WAIT_FOR_GAMESTART: begin
118         if (Logged_In == 1'b1 && B_load == 1'b1)
119             LogoutPulse = 1'b1;
120     end
121
122     S_GAMEPLAY: begin
123         Timer_Enable = 1'b1;
124         //RNG_Gen_Out = RNG_Gen_In;
125         RNG_Gen_Out  = ~RNG_Gen_In;
126         Load_P_Out   = Load_P_In;

```

Verilog file

```

ROMAuthentication.v  ROM_PSWD.v  GameController_Lab4.v  OneSecTimer.v  Lab4_KANDHALAM_Dharani
109  S_INIT: begin
110      // everything blocked
111  end
112
113  S_PASSED: begin
114      Timer_Reconfig = 1'b1;  // one-cycle pulse
115  end
116
117  S_WAIT_FOR_GAMESTART: begin
118      if (Logged_In == 1'b1 && B_load == 1'b1)
119          LogoutPulse = 1'b1;
120      end
121
122  S_GAMEPLAY: begin
123      Timer_Enable = 1'b1;
124      //RNG_Gen_Out  = RNG_Gen_In;
125      RNG_Gen_Out  = ~RNG_Gen_In;
126      Load_P_Out   = Load_P_In;
127  end
128
129  S_GAMEOVER: begin
130      ShowScore = 1'b1;
131      if (Logged_In == 1'b1 && B_load == 1'b1)
132          LogoutPulse = 1'b1;
133      end
134
135  default: begin
136      end
137  endcase
138  end
139
140  endmodule

```

Verification Module:

```

BinaryMathGameModule
MathGameModule  BinaryMathGameModule.v  My4BAdder.v  VerificationModule.v
1  // ECE6370 : ADD
2  // Author: Dharani Kandhalam - 1369
3  // Verification Module
4  // Description: It checks sum of player1+player2 == 15 or not
5  // if sum == 15 - we are turning on the Leftmost LED on Board, if not we are
6  // turning on RightMost LED on Board
7
8  module VerificationModule(Sum, LED_Left, LED_Right);
9      input  [3:0] Sum;
10     output      LED_Left, LED_Right; // one bit outputs
11
12
13     assign LED_Left  = (Sum == 4'b1111); // ON if sum = 15
14     assign LED_Right = (Sum != 4'b1111); // ON otherwise
15
16  endmodule

```

PlayerModule:

```
1 // ECE6370 : ADD
2 // Author: Dharani Kandharam - 1389
3 // Player Module
4 // Description: It takes a bit number from player and send it to
5 // 7segmentdecoder for Displaying on 7segment Display
6
7 module PlayerModule(PlayerSwitch, PlayerDisplay);
8   input [3:0] PlayerSwitch;
9   output [6:0] PlayerDisplay;
10  //output [6:0] D_out;
11
12  //wire [3:0] D_in;
13
14  My7SegmentDecoder_4To7 My7SegmentDisplay1(PlayerSwitch, PlayerDisplay);
15
16  //DoorOpener MyDoorOpener1(Person, FaceRecognized, CardRecognized, DLock_in);
17  //DecoderLock MyDecoderLock1(DLock_in, DLock_out);
18  //assign UnlockDoor = DLock_in;
19
20 endmodule
21
```

AdderModule:

```
BinaryMathGameModule.v My4BAdder.v VerificationModule.v PlayerModule.v
1 // ECE6370 : ADD
2 // Author: Dharani Kandharam - 1389
3 // 4 bit adder module
4 // Description: It adds 2 4bit numbers and stores that value into Sum
5
6 module My4BAdder(Num1, Num2, Sum);
7   input [3:0] Num1, Num2;
8   output [3:0] Sum;
9   reg [3:0] Sum;
10
11   always @(Num1, Num2)
12   begin
13     Sum = Num1 + Num2;
14   end
15
16 endmodule
17
```

7segmentDecoder Display Module:

```
1 // ECE6370 : ADD
2 // Author: Dharani Kandharam - 1389
3 // 7segment Decoder
4 // Description: It takes a bit number and convert that to 7 bit for
5 // displaying on 7 segment display
6
7 module My7SegmentDecoder_4To7(Decoder_In, Decoder_Out);
8   input [3:0] Decoder_In;
9   output [6:0] Decoder_Out;
10  reg [6:0] Decoder_Out;
11
12  always @(Decoder_In)
13  begin
14    case(Decoder_In)
15      4'b0000: begin Decoder_Out = 7'b1000000; end
16      4'b0001: begin Decoder_Out = 7'b0111100; end
17      4'b0010: begin Decoder_Out = 7'b0110010; end
18      4'b0011: begin Decoder_Out = 7'b0111000; end
19      4'b0100: begin Decoder_Out = 7'b0011001; end
20      4'b0101: begin Decoder_Out = 7'b0010010; end
21      4'b0110: begin Decoder_Out = 7'b0000010; end
22      4'b0111: begin Decoder_Out = 7'b1111000; end
23      4'b1000: begin Decoder_Out = 7'b0000000; end
24      4'b1001: begin Decoder_Out = 7'b0010000; end
25      4'b1010: begin Decoder_Out = 7'b0001000; end
26      4'b1011: begin Decoder_Out = 7'b0000011; end
27      4'b1100: begin Decoder_Out = 7'b1000110; end
28      4'b1101: begin Decoder_Out = 7'b0100001; end
29      4'b1110: begin Decoder_Out = 7'b0000011; end
30      4'b1111: begin Decoder_Out = 7'b0000110; end
31      default: begin Decoder_Out = 7'b1111111; end
32    endcase
33  end
34
35 endmodule
36
37
```

Random Number Generator:

```
1 // ECE6370
2 // Author: Dharani Kandharam - 1389
3 // Name of Module: RandomNumberGenerator
4 // Description: This module generates a 4-bit pseudo-random number using
5 // a 4-bit LFSR. The LFSR advances when rngEnable is HIGH.
6 // Comments/Log: Lab 4 bonus feature - LFSR-based RNG.
7
8 module RandomNumberGenerator(randomNum, rngEnable, clk, rst);
9
10     input clk, rst;
11     input rngEnable;
12     output [3:0] randomNum;
13
14     reg [3:0] randomNum;
15     wire feedback;
16
17     assign feedback = randomNum[3] ^ randomNum[2];
18
19     always @(posedge clk) begin
20         if (rst == 1'b0) begin
21             randomNum <= 4'b0001; // nonzero seed
22         end
23         else if (rngEnable == 1'b1) begin
24             randomNum <= {randomNum[2:0], feedback};
25         end
26     end
27
28 endmodule
```

OneSecTimer:

```
1 // ECE6370
2 // Author: Dharani Kandharam - 1389
3 // Name of Module: OneSecTimer
4 // Description: This module generates a one-second timeout pulse using
5 // three smaller timer stages. The first stage uses the Lab 4 LFSR-based
6 // 1 ms timer, followed by the 100 ms timer and the count-to-10 timer.
7 // Comments/Log: Updated for Lab 4 by replacing the regular 1 ms timer
8 // with the LFSR-based 1 ms timer.
9
10 module OneSecTimer(enable, clk, rst, OneSecTimeout);
11
12     input enable, clk, rst;
13     output OneSecTimeout;
14
15     wire msecTimeOut;
16     wire hundredmsecTimeOut;
17     wire [15:0] q_unused;
18
19     // Replace regular msecTimer with LFSR-based version
20     msecTimer_LFSR DUT_msecTimer_LFSR(enable, clk, rst, msecTimeOut, q_unused);
21
22     hundredmsecTimer DUT_hundredmsecTimer(msecTimeOut, clk, rst, hundredmsecTimeOut);
23
24     countTo10Timer DUT_countTo10Timer(hundredmsecTimeOut, clk, rst, OneSecTimeout);
25
26 endmodule
```

```
AccessController.v    testbench_RandomNumberGenerator.v    countTo10Timer.v    hundredmsecTimer.v
1 // ECE6370
2 // Author: Dharani Kandharam - 1389
3 // Name of Module: countTo10Timer
4 // Description: This module generates the final OneSecTimeout pulse by
5 // counting 10 enable pulses. When enable is HIGH, the counter increments
6 // from 0 to 9. Once the terminal count is reached, the counter resets
7 // and produces a one-clock-cycle OneSecTimeout pulse.
8 // Comments/Log: This module is used as the final stage in the nested
9 // OneSecTimer design for Lab 3.
10
11 module countTo10Timer(enable, clk, rst, OneSecTimeout);
12
13     input enable, clk, rst;
14     output OneSecTimeout;
15
16     reg OneSecTimeout;
17     reg [3:0] count;
18
19     always @(posedge clk) begin
20         if (rst==1'b0) begin
21             count <= 0;
22             OneSecTimeout <= 0;
23         end
24         else if (enable==1'b1) begin
25             if (count == 4'd9) begin
26                 count <= 0;
27                 OneSecTimeout <= 1;
28             end
29             else begin
30                 count <= count + 1;
31                 OneSecTimeout <= 0;
32             end
33         end
34         else
35             OneSecTimeout <= 0;
36     end
37
38 endmodule
```

```
AccessController.v  testbench_RandomNumberGenerator.v  countTo10Timer.v  hundredmsecTimer.v
1 // ECE6370
2 // Author: Dharani Kandhalam - 1389
3 // Name of Module: hundredmsecTimer
4 // Description: This module generates a hundredmsecTimeOut pulse by counting
5 // clock cycles when enable is HIGH. When the count reaches 99, the module
6 // resets the counter and produces a one-clock-cycle timeout pulse.
7 // The active-low reset clears both the counter and the timeout output.
8 // Comments/Log: This module is used as the intermediate timer stage in the
9 // nested OneSecTimer design for Lab 3.
10
11 module hundredmsecTimer(enable, clk, rst, hundredmsecTimeOut);
12
13     input enable, clk, rst;
14     output hundredmsecTimeOut;
15
16     reg hundredmsecTimeOut;
17     reg [6:0] count;
18
19     always @(posedge clk) begin
20         if (rst == 1'b0) begin
21             count <= 0;
22             hundredmsecTimeOut <= 0;
23         end
24         else if (enable == 1'b1) begin
25             if (count == 7'd99) begin
26                 count <= 0;
27                 hundredmsecTimeOut <= 1;
28             end
29             else begin
30                 count <= count + 1;
31                 hundredmsecTimeOut <= 0;
32             end
33         end
34         else
35             hundredmsecTimeOut <= 0;
36     end
37
38 endmodule
```

```

ROMAuthentication.v  ROM_PSWD.v  GameController_Lab4.v  OneSecTimer.v  Lab4_KANDHALAM_Dharani.qsf  Lab3
1  // ECE6370
2  // Author: Dharani Kandharam - 1389
3  // Name of Module: msecTimer_LFSR
4  // Description: This module generates a one-clock-cycle msecTimeOut pulse
5  // using a 16-bit LFSR. When enable is HIGH, the LFSR advances one state
6  // per clock cycle. If the next LFSR state matches the identified terminal
7  // value, the module generates the timeout pulse in the same cycle.
8  // Comments/Log: Final LFSR-based replacement for the regular 1 ms timer.
9
10 module msecTimer_LFSR(enable, clk, rst, msecTimeOut, q);
11
12     input enable, clk, rst;
13     output msecTimeOut;
14     output [15:0] q;
15
16     reg msecTimeOut;
17     reg [15:0] LFSR;
18
19     wire feedback;
20     wire [15:0] next_lfsr;
21
22     parameter TERMINAL_VALUE = 16'hb11f;
23
24     assign feedback = LFSR[15];
25
26     assign next_lfsr[0] = feedback;
27     assign next_lfsr[1] = LFSR[0];
28     assign next_lfsr[2] = LFSR[1] ^ feedback;
29     assign next_lfsr[3] = LFSR[2] ^ feedback;
30     assign next_lfsr[4] = LFSR[3];
31     assign next_lfsr[5] = LFSR[4] ^ feedback;
32     assign next_lfsr[6] = LFSR[5];
33     assign next_lfsr[7] = LFSR[6];
34     assign next_lfsr[8] = LFSR[7];
35     assign next_lfsr[9] = LFSR[8];
36     assign next_lfsr[10] = LFSR[9];
37     assign next_lfsr[11] = LFSR[10];
38     assign next_lfsr[12] = LFSR[11];
39     assign next_lfsr[13] = LFSR[12];
40     assign next_lfsr[14] = LFSR[13];
41     assign next_lfsr[15] = LFSR[14];
42

```

```
ROMAuthentication.v  ROM_PSWD.v  GameController_lab4.v  OneSecTimer.v  Lab4
40  assign next_lfsr[14] = LFSR[13];
41  assign next_lfsr[15] = LFSR[14];
42
43  assign q = LFSR;
44
45  always @(posedge clk) begin
46      if (rst == 1'b0) begin
47          LFSR <= 16'h0001;
48          msecTimeOut <= 1'b0;
49      end
50      else if (enable == 1'b1) begin
51          // protect against illegal all-zero state
52          if (LFSR == 16'h0000) begin
53              LFSR <= 16'h0001;
54              msecTimeOut <= 1'b0;
55          end
56          else if (next_lfsr == TERMINAL_VALUE) begin
57              LFSR <= 16'h0001;
58              msecTimeOut <= 1'b1;
59          end
60          else begin
61              LFSR <= next_lfsr;
62              msecTimeOut <= 1'b0;
63          end
64      end
65      else begin
66          msecTimeOut <= 1'b0;
67      end
68  end
69
70  endmodule
```

LFSR 1ms Module with Terminal Value:

C:/ADD_Projects/Lab4_KANDHALAM_Dharani/sim_1mslfsr/OnemsLFSRTimer.v (/testbench_msecLFSRTimer/DUT_lfsr1ms) - De

```
Ln#
1 // ECE6370
2 // Author: Dharani Kandharam - 1389
3 // Name of Module: msecTimer_LFSR
4 // Description: This module generates a one-clock-cycle msecTimeOut pulse
5 // using a 16-bit LFSR. When enable is HIGH, the LFSR advances one state
6 // per clock cycle. If the next LFSR state matches the identified terminal
7 // value, the module generates the timeout pulse in the same cycle.
8 // Comments/Log: Final LFSR-based replacement for the regular 1 ms timer.
9
10 module msecTimer_LFSR(enable, clk, rst, msecTimeOut, q);
11
12     input enable, clk, rst;
13     output msecTimeOut;
14     output [15:0] q;
15
16     reg msecTimeOut;
17     reg [15:0] LFSR;
18
19     wire feedback;
20     wire [15:0] next_lfsr;
21
22     parameter TERMINAL_VALUE = 16'hb11f;
23
24     assign feedback = LFSR[15];
25
26     // next-state logic
27     assign next_lfsr[0] = feedback;
28     assign next_lfsr[1] = LFSR[0];
29     assign next_lfsr[2] = LFSR[1] ^ feedback;
30     assign next_lfsr[3] = LFSR[2] ^ feedback;
31     assign next_lfsr[4] = LFSR[3];
32     assign next_lfsr[5] = LFSR[4] ^ feedback;
33     assign next_lfsr[6] = LFSR[5];
34     assign next_lfsr[7] = LFSR[6];
35     assign next_lfsr[8] = LFSR[7];
36     assign next_lfsr[9] = LFSR[8];
37     assign next_lfsr[10] = LFSR[9];
38     assign next_lfsr[11] = LFSR[10];
39     assign next_lfsr[12] = LFSR[11];
40     assign next_lfsr[13] = LFSR[12];
41     assign next_lfsr[14] = LFSR[13];
42     assign next_lfsr[15] = LFSR[14];
43
44     assign q = LFSR;
45
```

```
C:/ADD_Projects/Lab4_KANDHALAM_Dharani/sim_1mslfsr/OnemslFSRTimer.v (/testbench_msecFSRTimer)
Ln#
20 wire [15:0] next_lfsr;
21
22 parameter TERMINAL_VALUE = 16'hb11f;
23
24 assign feedback = LFSR[15];
25
26 // next-state logic
27 assign next_lfsr[0] = feedback;
28 assign next_lfsr[1] = LFSR[0];
29 assign next_lfsr[2] = LFSR[1] ^ feedback;
30 assign next_lfsr[3] = LFSR[2] ^ feedback;
31 assign next_lfsr[4] = LFSR[3];
32 assign next_lfsr[5] = LFSR[4] ^ feedback;
33 assign next_lfsr[6] = LFSR[5];
34 assign next_lfsr[7] = LFSR[6];
35 assign next_lfsr[8] = LFSR[7];
36 assign next_lfsr[9] = LFSR[8];
37 assign next_lfsr[10] = LFSR[9];
38 assign next_lfsr[11] = LFSR[10];
39 assign next_lfsr[12] = LFSR[11];
40 assign next_lfsr[13] = LFSR[12];
41 assign next_lfsr[14] = LFSR[13];
42 assign next_lfsr[15] = LFSR[14];
43
44 assign q = LFSR;
45
46 always @(posedge clk) begin
47     if (rst == 1'b0) begin
48         LFSR <= 16'h0001;
49         msecTimeOut <= 1'b0;
50     end
51     else if (enable == 1'b1) begin
52         if (next_lfsr == TERMINAL_VALUE) begin
53             LFSR <= 16'h0001; // restart sequence
54             msecTimeOut <= 1'b1; // pulse aligned with old timer
55         end
56         else begin
57             LFSR <= next_lfsr;
58             msecTimeOut <= 1'b0;
59         end
60     end
61     else begin
62         msecTimeOut <= 1'b0;
63     end
64 end
65
66 endmodule
```

Testbench after Terminal Value:

C:/ADD_Projects/Lab4_KANDHALAM_Dharani/sim_1mslfsr/Testbench_OnemsLFSRTimer.v (/testbench_msecLFSRTimer) - De

Ln#	
1	// ECE6370
2	// Author: Dharani Kandharam - 1389
3	// Name of Module: testbench_msecTimer_LFSR_compare
4	// Description: This testbench simulates the regular 1 ms timer and the
5	// final LFSR-based 1 ms timer side by side and verifies that both generate
6	// single-cycle timeout pulses at the same time.
7	// Comments/Log: Second run for exact timing comparison.
8	
9	module testbench_msecLFSRTimer;
10	
11	reg clk, rst, enable;
12	wire timeout_oldlms;
13	wire timeout_lfsrlms;
14	wire [15:0] lfsr_q;
15	
16	msecTimer DUT_oldlms (
17	enable(enable),
18	clk(clk),
19	rst(rst),
20	msecTimeOut(timeout_oldlms)
21	);
22	
23	msecTimer_LFSR DUT_lfsrlms (
24	enable(enable),
25	clk(clk),
26	rst(rst),
27	msecTimeOut(timeout_lfsrlms),
28	q(lfsr_q)
29	);
30	
31	always begin
32	clk = 1'b0; #10;
33	clk = 1'b1; #10;
34	end
35	
36	integer pulse_count;
37	
38	initial begin
39	rst = 1'b0;
40	enable = 1'b0;
41	pulse_count = 0;
42	

Ln#	
24	.enable(enable),
25	.clk(clk),
26	.rst(rst),
27	.msecTimeOut(timeout_lfsrlms),
28	.q(lfsr_q)
29	);
30	
31	always begin
32	clk = 1'b0; #10;
33	clk = 1'b1; #10;
34	end
35	
36	integer pulse_count;
37	
38	initial begin
39	rst = 1'b0;
40	enable = 1'b0;
41	pulse_count = 0;
42	
43	#25;
44	rst = 1'b1;
45	enable = 1'b1;
46	
47	#2500000;
48	\$stop;
49	end
50	
51	always @(posedge clk) begin
52	if (timeout_oldlms == timeout_lfsrlms) begin
53	pulse_count = pulse_count + 1;
54	\$display("Matched timeout pulse %0d at time=%0t", pulse_count, \$time);
55	
56	if (pulse_count == 2) begin
57	\$display("PASS: both timers matched for two full cycles.");
58	@(posedge clk);
59	@(posedge clk);
60	\$stop;
61	end
62	end
63	
64	if (timeout_oldlms ^ timeout_lfsrlms) begin
65	\$display("ERROR: timeout mismatch at time=%0t", \$time);
66	\$stop;
67	end
68	end
69	
70	endmodule

LFSR 1mse Timer module before Terminal Value:

```

C:/ADD_Projects/Lab4_KANDHALAM_Dharani/sim_1mslfsr/OnemslfsrTimer.v (/testbench_msecTimer_LFSR_findT
Ln#
1 // ECE6370
2 // Author: Dharani Kandhalam - 1389
3 // Name of Module: msecTimer_LFSR_findTerminal
4 // Description: This module advances a 16-bit LFSR one state per enabled
5 // clock cycle. It is used in side-by-side simulation with the regular
6 // 1 ms timer to identify the LFSR terminal value corresponding to the
7 // regular timer timeout.
8 // Comments/Log: Temporary version for terminal-state discovery.
9
10 module msecTimer_LFSR_findTerminal(enable, clk, rst, q);
11
12 input enable, clk, rst;
13 output [15:0] q;
14
15 reg [15:0] LFSR;
16 wire feedback;
17
18 assign feedback = LFSR[15];
19 assign q = LFSR;
20
21 always @(posedge clk) begin
22     if (rst == 1'b0) begin
23         LFSR <= 16'h0001; // nonzero seed
24     end
25     else if (enable == 1'b1) begin
26         LFSR[0] <= feedback;
27         LFSR[1] <= LFSR[0];
28         LFSR[2] <= LFSR[1] ^ feedback;
29         LFSR[3] <= LFSR[2] ^ feedback;
30         LFSR[4] <= LFSR[3];
31         LFSR[5] <= LFSR[4] ^ feedback;
32         LFSR[6] <= LFSR[5];
33         LFSR[7] <= LFSR[6];
34         LFSR[8] <= LFSR[7];
35         LFSR[9] <= LFSR[8];
36         LFSR[10] <= LFSR[9];
37         LFSR[11] <= LFSR[10];
38         LFSR[12] <= LFSR[11];
39         LFSR[13] <= LFSR[12];
40         LFSR[14] <= LFSR[13];
41         LFSR[15] <= LFSR[14];
42     end
43 end
44
45 endmodule

```

Testbench to find the LFSR terminal Value:

```

C:/ADD_Projects/Lab4_KANDHALAM_Dharani/sim_1mslfsr/Testbench_OnemsLFSRTerminalValue.v (/testbench_msecTim
Ln#
5 // LFSR-based timer side by side using the same clock, reset, and enable.
6 // It is used to identify the LFSR state present when the regular timer
7 // generates its timeout pulse.
8 // Comments/Log: First run for terminal-value discovery.
9 `timescale 1 ns/100 ps
10 module testbench_msecTimer_LFSR_findTerminal;
11
12 reg clk, rst, enable;
13 wire timeout_oldlms;
14 wire [15:0] lfsr_q;
15
16 msecTimer DUT_oldlms(enable, clk, rst, timeout_oldlms);
17
18 msecTimer_LFSR_findTerminal DUT_lfsrlms(enable, clk, rst, lfsr_q);
19
20 always begin
21     clk = 1'b0; #10;
22     clk = 1'b1; #10;
23 end
24
25 integer pulse_count;
26
27 initial begin
28     rst = 1'b0;
29     enable = 1'b0;
30     pulse_count = 0;
31
32     #25;
33     rst = 1'b1;
34     enable = 1'b1;
35
36     #2500000;
37     $stop;
38 end
39
40 always @(posedge clk) begin
41     if (timeout_oldlms == 1'b1) begin
42         pulse_count = pulse_count + 1;
43         $display("Regular lms pulse %0d at time=%0t, LFSR state = %h",
44             pulse_count, $time, lfsr_q);
45
46         if (pulse_count == 2)
47             $stop;
48     end
49 end
50
51 endmodule

```

TwoDigitTimer:

```

1  /* ECE6370
2  Author: Dharani Kandharam - 1389
3  Name of Module: OnedigitTimer
4
5  Description:
6  This module implements a one-digit decimal countdown timer for the Lab 3
7  game timer design. The module stores one BCD digit (0 to 9) and updates
8  its value on the rising edge of the clock. When the reconfig signal is
9  asserted, the digit is loaded with 9. When borrowDN is asserted, the module
10 attempts to decrement the current digit by 1.
11
12 If the digit value is greater than 0, it decrements normally. If the digit
13 value is 0, the module checks noBorrowUP to determine whether a higher digit
14 can still be decremented. If borrowing from the higher digit is allowed,
15 this digit reloads to 9 and generates a borrowUP pulse to the next higher
16 digit. If borrowing is not allowed, the digit remains at 0 and asserts
17 noBorrowDN to indicate that no further downward borrowing is possible.
18
19 This module is designed as a reusable building block for a scalable multi-
20 digit timer and does not use a finite state machine.
21 */
22
23 module OneDigitTimer(digit, Timer_enable, ReloadToNine, Timer_reconfig, clk, rst);
24
25     input clk, rst;
26     input Timer_enable, ReloadToNine, Timer_reconfig;
27     output [3:0] digit;
28
29     reg [3:0] digit;
30
31     always @(posedge clk) begin
32         if (rst == 1'b0) begin
33             digit <= 4'd0;
34         end
35         else begin
36             if (Timer_reconfig == 1'b1) begin
37                 digit <= 4'd9;
38             end
39             else if (Timer_enable == 1'b1) begin
40                 if (ReloadToNine == 1'b1) begin
41                     digit <= 4'd9;
42                 end
43                 else if (digit > 4'd0) begin
44                     digit <= digit - 4'd1;
45                 end
46                 else begin
47                     digit <= 4'd0;
48                 end
49             end
50         end
51     end
52 endmodule
53

```

Verilog file

```

1 // ECE6370
2 // Author: Dharami Kandharam - 1389
3 // Name of Module: twoDigitTimer
4 // Description: This module implements a two-digit decimal countdown timer
5 // using two cascaded one-digit timer modules. The ones digit receives the
6 // one-second pulse and generates a borrow signal to the tens digit when it
7 // rolls over from 9 to 0. The timer is initialized to 99 using the reconfig
8 // signal and counts down to 00. The timeOut signal is asserted when the
9 // timer reaches its final state.
10
11 module TwoDigitTimer(Tensdigit, Onesdigit, TimeOut, Timer_enable, Timer_reconfig, clk, rst);
12
13     input clk, rst;
14     input Timer_enable, Timer_reconfig;
15     output [3:0] Tensdigit, Onesdigit;
16     output TimeOut;
17
18     wire reload_ones;
19     wire enable_tens;
20     wire timer_done;
21
22     assign timer_done = (Tensdigit == 4'd0) && (Onesdigit == 4'd0);
23
24     assign reload_ones = Timer_enable && (Onesdigit == 4'd0) && (Tensdigit != 4'd0);
25     assign enable_tens = Timer_enable && (Onesdigit == 4'd0) && (Tensdigit != 4'd0);
26
27     OneDigitTimer ones_digit(Onesdigit, Timer_enable && !timer_done, reload_ones, Timer_reconfig, clk, rst);
28
29     OneDigitTimer tens_digit(Tensdigit, enable_tens, 1'b0, Timer_reconfig, clk, rst);
30
31     assign TimeOut = timer_done;
32
33 endmodule

```

```

1 timescale 1 ns/100 ps
2 module testbench_twoDigitTimer();
3
4     reg clk, rst;
5     reg Timer_enable, Timer_reconfig;
6     wire [3:0] Tensdigit, Onesdigit;
7     wire TimeOut;
8
9     //clock
10    always
11    begin
12        clk = 1'b0;
13        #10;
14        clk = 1'b1;
15        #10;
16    end
17
18    TwoDigitTimer DUT_MyTwoDigitTimer(Tensdigit, Onesdigit, TimeOut, Timer_enable, Timer_reconfig, clk, rst);
19
20    initial
21    begin
22        rst = 1'b1;
23        Timer_reconfig = 1'b0;
24        Timer_enable = 1'b0;
25
26        @(posedge clk);
27        @(posedge clk);
28
29        //Reset
30        #5 rst = 1'b0;
31        @(posedge clk);
32        @(posedge clk);
33        @(posedge clk);
34        @(posedge clk);
35
36        //release reset
37        #5 rst = 1'b1;
38        @(posedge clk);
39        @(posedge clk);
40
41        //reconfigure timer to 99
42        #5 Timer_reconfig = 1'b1;
43        @(posedge clk);
44        #5 Timer_reconfig = 1'b0;
45        @(posedge clk);
46        @(posedge clk);
47
48        //99 to 98
49        #5 Timer_enable = 1'b1;
50        @(posedge clk);
51        #5 Timer_enable = 1'b0;
52        @(posedge clk);
53

```

```
testbench_OneSecTimer.v  OneDigitTimer.v  TwoDigitTimer.v  testbench_TwoDigitTimer.v
52 @ (posedge clk);
53
54 //98 to 97
55 #5 Timer_enable = 1'b1;
56 @ (posedge clk);
57 #5 Timer_enable = 1'b0;
58 @ (posedge clk);
59
60 //97 to 96
61 #5 Timer_enable = 1'b1;
62 @ (posedge clk);
63 #5 Timer_enable = 1'b0;
64 @ (posedge clk);
65
66 //96 to 95
67 #5 Timer_enable = 1'b1;
68 @ (posedge clk);
69 #5 Timer_enable = 1'b0;
70 @ (posedge clk);
71
72 //95 to 94
73 #5 Timer_enable = 1'b1;
74 @ (posedge clk);
75 #5 Timer_enable = 1'b0;
76 @ (posedge clk);
77
78 //94 to 93
79 #5 Timer_enable = 1'b1;
80 @ (posedge clk);
81 #5 Timer_enable = 1'b0;
82 @ (posedge clk);
83
84 //93 to 92
85 #5 Timer_enable = 1'b1;
86 @ (posedge clk);
87 #5 Timer_enable = 1'b0;
88 @ (posedge clk);
89
90 //92 to 91
91 #5 Timer_enable = 1'b1;
92 @ (posedge clk);
93 #5 Timer_enable = 1'b0;
94 @ (posedge clk);
95
96 //91 to 90
97 #5 Timer_enable = 1'b1;
98 @ (posedge clk);
99 #5 Timer_enable = 1'b0;
100 @ (posedge clk);
101
102 //90 to 89 rollover check
103 #5 Timer_enable = 1'b1;
104 @ (posedge clk);
```

Verilog file

```
testbench_OneSecTimer.v  OneDigitTimer.v  TwoDigitTimer.v  testbench_TwoDigitTimer.v
94      @ (posedge clk);
95
96      //91 to 90
97      #5 Timer_enable = 1'b1;
98      @ (posedge clk);
99      #5 Timer_enable = 1'b0;
100     @ (posedge clk);
101
102     //90 to 89 rollover check
103     #5 Timer_enable = 1'b1;
104     @ (posedge clk);
105     #5 Timer_enable = 1'b0;
106     @ (posedge clk);
107
108     //few more counts
109     #5 Timer_enable = 1'b1;
110     @ (posedge clk);
111     #5 Timer_enable = 1'b0;
112     @ (posedge clk);
113
114     #5 Timer_enable = 1'b1;
115     @ (posedge clk);
116     #5 Timer_enable = 1'b0;
117     @ (posedge clk);
118
119     #5 Timer_enable = 1'b1;
120     @ (posedge clk);
121     #5 Timer_enable = 1'b0;
122     @ (posedge clk);
123
124     $stop;
125 end
126
127 endmodule
```

ScoreBoard:

```

1 // ECE6370
2 // Author: Dharani Kandhalam - 1389
3 // Name of Module: Scoreboard
4 // Description: This module implements a two-digit BCD scoreboard.
5 // It increments once for each valid correct load event and resets to 00
6 // when reconfig is asserted or reset is active.
7
8 module Scoreboard(scoreTens, scoreOnes, load_pulse, correct_led, timeout, reconfig, clk, rst);
9
10     input clk, rst;
11     input load_pulse, correct_led, timeout, reconfig;
12     output [3:0] scoreTens, scoreOnes;
13
14     reg [3:0] scoreTens, scoreOnes;
15     reg load_pulse_d;
16
17     always @(posedge clk) begin
18         if (rst == 1'b0) begin
19             scoreTens <= 4'd0;
20             scoreOnes <= 4'd0;
21             load_pulse_d <= 1'b0;
22         end
23         else begin
24             load_pulse_d <= load_pulse;
25
26             if (reconfig == 1'b1) begin
27                 scoreTens <= 4'd0;
28                 scoreOnes <= 4'd0;
29             end
30             else if ((load_pulse_d == 1'b1) && (correct_led == 1'b1) && (timeout == 1'b0)) begin
31                 if ((scoreTens == 4'd9) && (scoreOnes == 4'd9)) begin
32                     scoreTens <= 4'd9;
33                     scoreOnes <= 4'd9;
34                 end
35                 else if (scoreOnes == 4'd9) begin
36                     scoreOnes <= 4'd0;
37                     scoreTens <= scoreTens + 4'd1;
38                 end
39                 else begin
40                     scoreOnes <= scoreOnes + 4'd1;
41                 end
42             end
43         end
44     end
45
46 endmodule

```

All the necessary files and modules for the project are neatly organized in the folder named src in main project folder. Explore the folder form complete source code, sim for testbenches, and doc for documentation to ensure a smooth review experience.